Neurocomputing 643 (2025) 130400

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Survey paper

Spatio-temporal prediction using graph neural networks: A survey

Vincenzo Capone *, Angelo Casolaro, Francesco Camastra

Department of Science and Technology, Parthenope University of Naples, Centro Direzionale Isola C4, Naples, 80143, Italy

ARTICLE INFO

Communicated by M. Wu

Keywords:
Spatio-temporal graph neural networks
Spatio-temporal series
Spatio-temporal graphs
Deep learning
Benchmarking

ABSTRACT

The analysis of spatial time series is increasingly relevant as spatio-temporal data are becoming widespread due to the ever-growing diffusion of data acquisition devices. Spatio-temporal prediction is crucial for grasping insights on spatio-temporal dynamics in diverse domains. In many cases, spatio-temporal data can be effectively represented using graphs, thus making Graph Neural Networks the most sounding deep learning architecture for the modelling of spatio-temporal series. The aim of the work is to provide a self-consistent and thorough overview on Graph Neural Networks for spatio-temporal prediction, giving a taxonomy of the diverse approaches proposed in the literature. Moreover, attention is paid to the description of the most used benchmarks and metrics in different real-world spatio-temporal domains and to the discussion of the main drawbacks of spatio-temporal Graph Neural Networks. Furthermore, unlike other similar works on deep learning, statistical methods for spatio-temporal modelling are briefly surveyed in this work. Finally, insights on future developments of Graph Neural Networks for spatio-temporal prediction are suggested.

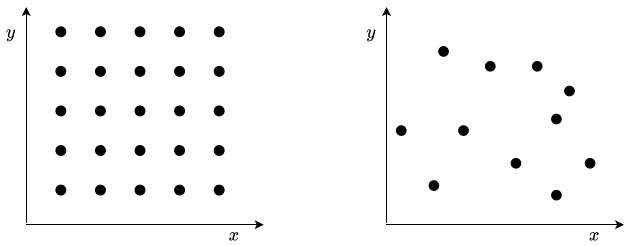
1. Introduction

In many scientific domains (e.g., environmental sciences [1–3], health and medicine [4–6], astronomy [7–9]), measurements often have associated spatial and temporal information, indicating where and when each observation was acquired. Several methods have been proposed to model *spatio-temporal data* leveraging additional spatial and temporal information. Since the early 1980s, in statistics, there have been several efforts to formally and theoretically describe these data [10,11]. However, only recently, spatio-temporal modelling has been tackled using *machine learning* and *deep learning* techniques [12, 13]. When modelling spatio-temporal data, a relevant task is *spatio-temporal prediction*, and, in particular, *spatio-temporal forecasting*, consisting in predicting future spatial values starting from past spatio-temporal information. The task is crucial in many applicative domains. In traffic [14–16], spatio-temporal forecasting allows the prediction of future vehicular traffic flow in different roads and the implementation of proper mitigation measures. In energy production [17–19], spatio-temporal forecasting is useful to predict power production over a network of power stations (e.g., wind farms) and power demand at diverse geographical areas. In the environmental domain [20–22], spatio-temporal prediction allows the monitoring of environmental variables, such as wind speed, or the spread of primary air pollutants in specific geographical areas. Moreover, spatio-temporal forecasting has an important role in healthcare [23,24], as it allows predicting the spread of epidemics over neighbouring areas and implementing proper clinical containment measures.

Among the many forms in which spatio-temporal data can occur [12], the so-called *spatio-temporal raster data* is the most common. This type of spatio-temporal data can be *regularly* or *irregularly sampled* in both space and time. Spatial sampling is particularly important, since the way that raster spatio-temporal data is spaced, makes some deep learning models more suitable than others for the processing of this type of data. Fig. 1 shows two examples of spatio-temporal data. On the left, observations are fixed on a regular grid and the horizontal/vertical distances between two adjacent locations are constant. On the right, observations are acquired at some given locations, but the distance between two adjacent locations depends on the actual locations themselves. Regularly-spaced spatio-temporal data can be organised on a regular three-dimensional lattice, with one dimension being time and the remaining two being space, whereas irregularly-spaced data are better represented as *time series of spatial graphs*, whose edges encode neighbouring relations among observations. Regularly-spaced data can be processed using deep learning architectures that leverage such spatial regularity, e.g., spatio-temporal CNNs [25,26], Convolutional LSTMs [27,28] and Transformer models [29–32]. Although these architectures may be adapted to process irregularly-spaced data as well, this type of spatio-temporal data is better processed using extensions

* Corresponding author.
E-mail addresses: vincenzo.capone002@studenti.uniparthenope.it (V. Capone), angelo.casolaro001@studenti.uniparthenope.it (A. Casolaro), francesco.camastra@uniparthenope.it (F. Camastra).

https://doi.org/10.1016/j.neucom.2025.130400
Received 3 December 2024; Received in revised form 22 March 2025; Accepted 30 April 2025
Available online 15 May 2025



(a) Regularly-spaced data (b) Irregularly-spaced data

Fig. 1. Graphical examples of regularly (Fig. 1(a)) and irregularly (Fig. 1(b)) spaced raster data. In the former case, a neighbourhood is implicitly defined by the spatial grid containing the observations; in the latter, it can be encoded using edges of a graph.

of *Graph Neural Networks* (GNNs) to the spatio-temporal domain, the so-called *Spatio-Temporal GNNs*.

Most surveys covering spatio-temporal prediction using GNNs are focused on specific domains, e.g., transportation [33–35]. There is only a single work that provides a generic and wide coverage of spatio-temporal GNN approaches [36]. Although each reviewed paper is analysed in detail, the work does not present a taxonomy in which the diverse contributions can be classified and does not provide a comprehensive review of the GNN theory and insights that make the work really helpful for potential readers. In this work, a survey on spatio-temporal prediction using GNNs is presented, paying attention to how GNNs can be designed for spatio-temporal prediction. In particular, the key contributions of the work are:

- a detailed description of the used GNN approaches is provided;
- a taxonomy of the diverse GNN approaches is proposed;
- advantages and drawbacks of spatio-temporal GNNs are identified, and possible improvements are suggested;
- an overview on the most common applicative fields in which spatio-temporal GNNs are used, along with the most popular datasets and metrics, is given;
- a specific section of the work is devoted to the description of statistical methods for spatio-temporal prediction, unlike existing surveys on spatio-temporal prediction using machine learning.

The work is organised as follows. Section 2 gives an introduction to statistical methods for spatio-temporal prediction. Section 3 provides an overview of the deep learning methodologies and techniques that are also used as building blocks in spatio-temporal GNNs. Section 4 offers an in-depth discussion on spatio-temporal graph neural networks; underlining methodologies, techniques, challenges, applications and datasets. Section 5 discusses the main limitations of spatio-temporal GNNs. Finally, Section 6 draws some conclusions on the use of GNNs for spatio-temporal modelling tasks.

2. Statistical modelling methods

Spatio-temporal Graph Neural Networks are naturally suited for the processing of irregularly-spaced spatio-temporal data. This particular type of spatio-temporal data can also be processed using more traditional statistical methodologies. In this context, a generic spatio-temporal process can be written as:

$$y(\vec{s},t) \qquad \forall \vec{s} \in \mathcal{S}, t \in \mathcal{T}, \tag{1}$$

where $\mathcal{S}$ and $\mathcal{T}$ are the *spatial* and *temporal domains* of the spatio-temporal process. In statistical literature, there are two main spatio-temporal modelling approaches, the *descriptive* and the *dynamic* ones [37], both trying to capture the statistical relationships between spatio-temporal variables.

Descriptive approaches typically model spatio-temporal processes in terms of a *mean function* and a *covariance function*. Following a descriptive approach, a spatio-temporal process can be formalised as follows [38]:

$$y(\vec{s},t) = f_\beta(\vec{s},t) + \eta(\vec{s},t), \tag{2}$$

where the function $f_\beta(\vec{s},t)$, with parameters β, is the mean of $y(\vec{s},t)$, which can also depend on some covariate variables $\vec{z}(\vec{s},t)$, i.e., $f_\beta(\vec{s},t) = f_\beta(\vec{z}(\vec{s},t))$. $\eta(\vec{s},t)$ is a spatio-temporal random process, usually with a mean function $\mu(\vec{s},t) = 0$ and a covariance function $C(\vec{s},t;\vec{s}',t')$ that has to be estimated. The estimation of such covariance function is commonly based on *Gaussian Processes* and *Markov Random Fields* [38].

A relevant feature of descriptive statistical models is that, once their parameters have been estimated, they can be used to perform not only predictions but also to quantify the *uncertainty* on the obtained prediction.

Dynamic (or *conditional*) modelling approaches seek to capture statistical dependencies within the spatio-temporal process by modelling its dynamics, that is, by capturing how the values at a given location $\vec{s}$ and at a given time t are influenced by the values observed at other locations in the past. Some of the approaches that fall in this category are the spatio-temporal extensions of the *autoregressive* methods commonly adopted for non-spatial time series.

2.1. Spatio-temporal autoregressive approaches

A univariate spatial time series can be generated by a spatio-temporal process where each location can be seen as an univariate time series. A spatial time series can be organised as a multivariate time series $\{\vec{y}_t\}_{t=1}^T, \vec{y}_t \in \mathbb{R}^N$, where each dimension of the multivariate series corresponds to a location and N is the number of spatial locations being observed.

In analogy with traditional time series [39], spatial time series can be modelled using linear *Spatio-Temporal AutoRegressive* (STAR) models [10,11,40], denoted as STAR($p_{\vec{\lambda}}$):

$$\vec{y}_t = \vec{\epsilon}_t + \sum_{k=1}^{p}\sum_{l=0}^{\lambda_k} \phi_{k,l} U_{k,l} \vec{y}_{t-k}, \tag{3}$$

where p is the *model order* of STAR, namely how many past samples are required to model the spatio-temporal series adequately, and $\vec{\lambda} = \{\lambda_1, \dots, \lambda_p\}$ are the *spatial orders* λ_k at different time lags $k \in [1,p]$. $\vec{\epsilon}_t$ is a zero-mean, normally distributed noise vector; $U_{k,l} \in \mathbb{R}^N \otimes \mathbb{R}^N$ are known *spatial weighing matrices*, fixed by the modeller to account for the spatial relationships between variables over different locations at time lag k. They are usually defined in terms of *inverse distances*, following the idea that spatially close locations influence each other more than spatially far locations.

The original STAR formulation can be modified to account for the influence that *exogenous variables*, i.e., variables that are external to the autoregressive process (e.g., wind speed and direction on the diffusion of atmospheric pollutants), have on the target variable. The resulting model is commonly referred to as *Spatio-Temporal AutoRegressive* model with *eXogenous inputs* (*STARX*) [41], which can be formulated as[1]:

$$\vec{y}_t = \vec{\epsilon}_t + \sum_{k=1}^{p} U_k \Lambda_k \vec{y}_{t-k} + \sum_{k=0}^{h} \Psi_k \vec{z}_{t-k}. \tag{4}$$

$\{\vec{y}_t\}_{t=1}^T, \vec{y}_t \in \mathbb{R}^N$ is the multivariate series obtained from the aggregation of the univariate series at the N spatial locations; $\{\vec{z}_t\}_{t=1}^T, \vec{z}_t \in \mathbb{R}^M$ is the series of M exogenous variables that the model should consider;

[1] The formulation reported in Eq. (4) follows the historical definition proposed by David S. Stoffer [41]. However, in [41], the model is referred to as STARMAX, although there is no moving average component. For this reason, in this work, the model is presented as a STARX model.

$U_k \in \mathbb{R}^N \otimes \mathbb{R}^N$ is a pre-defined spatial weighing matrix at time lag k; $\Lambda_k \in \mathbb{R}^N \otimes \mathbb{R}^N$ is a *diagonal space–time transition matrix* that has to be estimated for each time lag k; $\Psi_k \in \mathbb{R}^N \otimes \mathbb{R}^M$ is a *covariate regression matrix* modelling the effect that the exogenous variables have on the target variables $\vec{y}_t$ at time lag k. It is worthwhile to remark how the index k in the covariates component starts from 0 instead of 1, indicating that the influence of covariates at the current time step t is also considered. In analogy with the STAR model, p is the autoregressive model order of STARX, while h denotes the model order of the exogenous component.

For the sake of completeness, variations of the aforementioned autoregressive models integrating a moving average component also exist [10,11,40,42].

All the aforementioned statistical models are linear and hence they cannot describe the behaviour of many real spatial time series that are generated by nonlinear dynamical systems [11]. *Nonlinear Spatio-Temporal AutoRegressive* (*NSTAR*) models can be formulated as:

$$\vec{y}_t = F(\vec{y}_{t-1}, \vec{y}_{t-2}, \dots, \vec{y}_{t-p}) + \vec{\epsilon}_t, \tag{5}$$

where p is the *model order* and $F(\cdot)$ is the *skeleton*, a generic nonlinear function modelling the spatio-temporal series. Exogenous variables can be included along with lagged series values, obtaining a *Nonlinear Spatio-Temporal AutoRegressive* model with *eXogenous inputs* (*NSTARX*):

$$\vec{y}_t = F(\vec{y}_{t-1}, \vec{y}_{t-2}, \dots, \vec{y}_{t-p}, \vec{z}_t, \vec{z}_{t-1}, \vec{z}_{t-2}, \dots, \vec{z}_{t-h}) + \vec{\epsilon}_t, \tag{6}$$

A promising statistical frameworks in which to integrate nonlinear dynamics is that of *Hierarchical Dynamical Spatio-Temporal Models* (*H-DSTM*) [11,37,43]. Hierarchical models assume that there is *uncertainty* about the true spatio-temporal process of interest $\vec{y}_t$ due to a discrepancy between $\vec{y}_t$ and the actually observed samples $\vec{z}_t$. H-DSTMs leverage probability theory to model, under Markovian assumptions, y_t while accounting for such an uncertainty. This is done through a hierarchy of three models: a *data model* $\vec{z}_t = \mathcal{H}(\vec{y}_t, \theta_d, \vec{\epsilon}_t)$, mapping the latent process $\vec{y}_t$ to an observation $\vec{z}_t$, a *process model* $\vec{y}_t = \mathcal{M}(\vec{y}_{t-1}, \theta_p, \vec{\eta}_t)$, autoregressively describing the evolution of the latent process, and a *parameter model*, yielding parameters θ_d and θ_p for the two other models. Integrating nonlinear dynamics within an H-DSTM framework is easily done by defining the process model, i.e., the function $\mathcal{M}(\cdot)$, as nonlinear.

The aforementioned statistical methods have two major limitations. First, they do not generally provide data-driven learning procedure for the model parameters [44]. Even when such a procedure is present, it is usually limited to the use of particular statistical inference methods [38,44]. Second, they may fail to fully grasp complex nonlinear relations among irregularly-spaced data. To cope with these limitations, the use of alternative deep learning techniques for spatio-temporal prediction is suitable. In particular, Spatio-Temporal Graph Neural Networks (STGNNs) compare favourably with the aforementioned statistical methods. On one side, the graph-modelling capability of STGNNs makes them naturally suited for the processing of irregularly spaced spatio-temporal data. On the other side, since they are deep learning models, STGNNs can better model complex nonlinear dependencies.

3. Deep learning recalls for STGNNs

STGNNs leverage other existing deep learning architectures, such as convolutional or recurrent neural networks, to process spatial and temporal relations in spatio-temporal data. To provide a self-consistent structure to the survey, this section recalls the deep learning architectures that are used within the STGNNs' framework. Section 3.1 presents Convolutional Neural Networks and one of its variations, i.e., Temporal Convolutional Networks. Section 3.2 discusses Recurrent Neural Networks and its variants, namely, Long Short-Term Memory networks, Gated Recurrent Units and Echo State Networks. Finally, Section 3.3 recalls the Transformer architecture and attention mechanisms. Fig. 2 provides an overview of the main deep learning methodologies adopted in STGNNs, highlighting the type of processing for which each methodology is used.

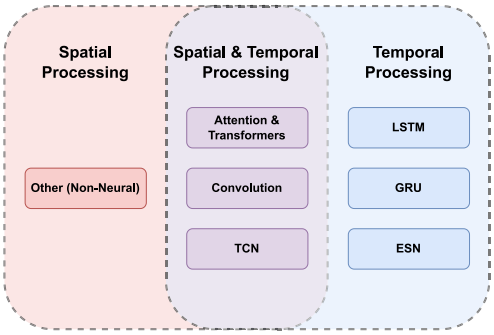


Fig. 2. An overview on the usage in STGNNs of deep learning methodologies, categorised by the type of processing they are used for. Methodologies used for spatial processing are highlighted in red; in blue, there are the methodologies used for temporal processing; in purple there are methodologies used for both spatial and temporal processing.

3.1. Convolutional Neural Networks

Convolutional Neural Networks (CNNs) leverage *convolutions* to extract spatial and temporal features. The use of convolutions gives CNNs three distinctive characteristics: *local connectivity*, *weight sharing* and *translation equivariance* [45]. Depending on the input's nature, CNNs are based on different types of convolutions: *mono-dimensional*, *bi-dimensional* and *three-dimensional* convolutions. Spatio-temporal GNNs use convolutions to model temporal dependencies among input samples; in particular, 1D convolutions and temporal convolutions are the most popular and they are briefly described in the following.

Given an input sequence $X = \{\vec{x}_t\}_{t=1}^T$, with $\vec{x}_t \in \mathbb{R}^D$, and a kernel $W \in \mathbb{R}^Q \otimes \mathbb{R}^D$, where Q is the kernel size, 1D convolution between W and X can be defined as:

$$y_t = (W * X)_t = \sum_{q=0}^{Q-1} \langle \vec{w}_q, \vec{x}_{t-q+\lfloor Q/2 \rfloor} \rangle \qquad \forall t, \tag{7}$$

where $(W * X)$ is the convolution operator, $\vec{w}_q \in \mathbb{R}^D$ is an element of the kernel, that is, the qth row of the matrix W, and $\langle \cdot, \cdot \rangle$ denotes the dot-product.

Standard 1D convolution may not be the best choice for modelling temporal dependencies that span several non-contiguous time steps, since it has a limited *receptive field*. When processing the tth time step, 1D convolution considers the element $\vec{x}_t$ and a few contiguous elements in the close past and future. Normally, the receptive field can be increased by either using kernels with a larger size Q or by a composition of multiple stacked convolution layers.

3.1.1. Temporal Convolutional Networks

When modelling temporal data, a better choice is the use of *causal dilated convolutions*, which are the building blocks of *Temporal Convolutional Networks* (TCNs) [46]. While standard 1D convolutions apply the kernel over elements from both the past and the future, causal dilated convolutions consider only past elements. Furthermore, in standard 1D convolutions the kernel is always applied to contiguous elements, while in causal dilated convolutions a fixed step between element pairs is considered with the introduction of a *positive dilation factor* $d \in \mathbb{N}$. A 1D causal dilated convolution with dilation factor d is described in Eq. (8):

$$y_t = (W * X)_t = \sum_{q=0}^{Q-1} \langle \vec{w}_q, \vec{x}_{t-dq} \rangle \qquad \forall t \in [1, T]. \tag{8}$$

TCNs are made by stacking multiple dilated convolution layers and using a different dilation factor at each layer. The use of causal dilated convolutions gives TCNs two main advantage: they can model temporal dependencies at different time scales, by using different dilation factors

at each layers, and they can capture dependencies with elements from the far past using a smaller number of stacked layers than what would be required with standard convolutions.

3.2. Recurrent Neural Networks

Recurrent Neural Networks (RNNs) [47] are deep learning architectures generally employed to model temporal relations among samples in a sequence. In spatio-temporal GNNs, RNNs are used to capture temporal dependencies. In particular, recurrent modules in spatio-temporal GNNs are mostly based on Long Short-Term Memory units, Gated Recurrent Units and Echo State Networks.

One of the earliest RNN formulations is *Elman's* [48], or *vanilla RNN*. A vanilla RNN layer processes an input sequence $X = \{\vec{x}_t\}_{t=1}^{T}$ one element $\vec{x}_t \in \mathbb{R}^D$ at a time, by updating, at each time step, an hidden representation $\vec{h}_t \in \mathbb{R}^K$, according to the following equation:

$$\vec{h}_t = g\left(W\vec{h}_{t-1} + U\vec{x}_t + \vec{b}\right) \qquad \forall t \in [1, T], \tag{9}$$

where $W \in \mathbb{R}^K \otimes \mathbb{R}^K$, $U \in \mathbb{R}^K \otimes \mathbb{R}^D$ and $\vec{b} \in \mathbb{R}^K$ are learnable hidden-hidden weights matrix, input-hidden weights matrix, and bias vector, respectively, K is the dimensionality of the hidden RNN space and $g(\cdot)$ is a generic nonlinear function, e.g., the hyperbolic tangent.

Vanilla RNNs, trained with backpropagation, can suffer of *vanishing* or *exploding gradients* [45]. Therefore, two variants, *Long Short-Term Memory* (LSTM) networks [49] and *Gated Recurrent Units* (GRUs) [50], are preferred as they are more stable during training.

3.2.1. Long Short-Term Memory networks

A LSTM unit has three gates: *input*, *forget* and *output*. LSTM's input vector $\vec{i}_t$ is computed from the current input $\vec{x}_t$ and the previous LSTM output $\vec{h}_{t-1}$ as follows:

$$\vec{i}_t = \beta(W_i\vec{x}_t + U_i\vec{h}_{t-1} + \vec{b}_i). \tag{10}$$

where $\beta(\cdot)$ can be any *sigmoidal function* and $\{W_i, U_i, \vec{b}_i\}$ is a set of parameters. Then, the three gates are computed as:

$$\begin{aligned}
\vec{g}_t &= \sigma(W_g\vec{x}_t + U_g\vec{h}_{t-1} + \vec{b}_g),\\
\vec{f}_t &= \sigma(W_f\vec{x}_t + U_f\vec{h}_{t-1} + \vec{b}_f),\\
\vec{o}_t &= \sigma(W_o\vec{x}_t + U_o\vec{h}_{t-1} + \vec{b}_o),
\end{aligned} \tag{11}$$

where $\sigma(\cdot)$ is the logistic function, $\vec{g}_t$, $\vec{f}_t$, $\vec{o}_t$ are the input, forget, and output gates, respectively, and $\{W_g, U_g, \vec{b}_g\}$, $\{W_f, U_f, \vec{b}_f\}$, $\{W_o, U_o, \vec{b}_o\}$ are the corresponding sets of parameters. Then, the LSTM's *inner state* $\vec{c}_t$ is updated by a linear combination of $\vec{i}_t$ and the *previous inner state* $\vec{c}_{t-1}$:

$$\vec{c}_t = \vec{g}_t \odot \vec{i}_t + \vec{f}_t \odot \vec{c}_{t-1}. \tag{12}$$

where $\odot$ is the *element-wise product*. Finally, the output $\vec{h}_t$ of a LSTM layer is defined as:

$$\vec{h}_t = \vec{o}_t \odot \tanh(\vec{c}_t). \tag{13}$$

3.2.2. Gated Recurrent Units

A GRU is composed of two gates: a *reset gate* $\vec{r}_t$ and an *update gate* $\vec{u}_t$, weighing the influence of the previous GRU output $\vec{h}_{t-1}$ and the current input $\vec{x}_t$ on the current GRU output $\vec{h}_t$, respectively, as follows:

$$\begin{aligned}
\vec{r}_t &= \sigma\left(W_r\vec{h}_{t-1} + U_r\vec{x}_t + \vec{b}_r\right),\\
\vec{u}_t &= \sigma\left(W_u\vec{h}_{t-1} + U_u\vec{x}_t + \vec{b}_u\right).
\end{aligned} \tag{14}$$

$\{W_r, U_r, \vec{b}_r\}$ and $\{W_u, U_u, \vec{b}_u\}$ are sets of parameters for each of the two gates; $\sigma(\cdot)$ is a sigmoid activation function. The reset gate $\vec{r}_t$ is used to compute an intermediate output $\vec{h}'_t$:

$$\vec{h}'_t = \tanh\left(W(\vec{r}_t \odot \vec{h}_{t-1}) + U\vec{x}_t + +\vec{b}\right), \tag{15}$$

where $\{W, U, \vec{b}\}$ is an additional set of parameters and $\odot$ represents the element-wise product. The final output $\vec{h}_t$ of the GRU is then computed using the update gate as follows:

$$\vec{h}_t = \vec{u}_t \odot \vec{h}_{t-1} + (\vec{e} - \vec{u}_t) \odot \vec{h}'_t, \tag{16}$$

where $\vec{e}$ is a column vector whose elements are all equal to 1.

3.2.3. Echo State Networks

A further variation of the vanilla RNN is the *Echo State Network* (ESN) [51]. ESNs fall into a *reservoir computing* framework, where the latent representation of the input is computed from a *fixed reservoir*, which corresponds to the recurrent layer. ESNs follow the same update equations of a vanilla RNN layer (Eq. (9)), but the matrices W and U, are *fixed at random*. Furthermore, a given sparsity level is imposed on W and U by zeroing some of their elements. ESNs have two main advantages: they do not generally suffer from vanishing and exploding gradients and their training is much faster compared with LSTM networks and GRUs.

3.3. Transformers and attention mechanisms

The original *Transformer* architecture [52] follows an encoder–decoder design for sequence-to-sequence transduction. Transformers can grasp long-term dependencies thanks to *attention mechanisms* [52], allowing models to *focus their attention* on specific parts of the input sequence, depending on the information being processed.

An attention mechanism works on a triplet of elements: a *query* vector $\vec{q}$, a set of N *value* vectors $\vec{v}_i$, each paired with a *key* vector $\vec{k}_i$. The query represents the information being processed by the model; keys and values represent the input on which the model has to focus its attention. Keys and values are usually the same vectors. For a given query, an attention mechanism produces an output $\vec{y}$, that is a linear combination of the N value vectors:

$$\vec{y} = \sum_{i=1}^{N} \alpha_i \vec{v}_i. \tag{17}$$

The weights α_i are obtained by a *compatibility function* between the query $\vec{q}$ and the corresponding key $\vec{k}_i$. Transformers adopt the *scaled dot-product attention*, i.e., a dot-product between query and key, scaled by the square root of their dimensionality D. Firstly, an unnormalised score e_i is computed for each key as:

$$e_i = \frac{\vec{q}^{\top}\vec{k}_i}{\sqrt{D}}. \tag{18}$$

Then, these scores are normalised via a softmax layer to obtain the final weights α_i:

$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}. \tag{19}$$

When queries are multiple, an attention mechanism is applied to each one of them separately, yielding multiple output vectors as follows:

$$Y = \text{Attention}(K, V, Q) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{D_k}}\right)V, \tag{20}$$

where $Q \in \mathbb{R}^M \otimes \mathbb{R}^D$ is a matrix whose rows contain query vectors. Similarly, keys and values are organised as rows into the matrices $K \in \mathbb{R}^N \otimes \mathbb{R}^D$ and $V \in \mathbb{R}^N \otimes \mathbb{R}^D$, respectively. The attention output $Y \in \mathbb{R}^M \otimes \mathbb{R}^D$ is a matrix whose ith row contains the output vector for the ith query. Moreover, attention is distinguished into *self-attention* and *cross-attention*. In the former, queries and keys/values are elements of the same sequence; in the latter, they are vectors belonging to two different sequences.

Transformers use the so-called *Multi-Head Attention* (MHA), where keys, values and queries are linearly projected H separate times by learned projection matrices to spaces of dimensions D_k, D_v and D_k

respectively. Then, a scaled dot-product attention is applied to each of the projections and the results are concatenated together and re-projected to the original space. Each projection-attention pair defines an *attention head* h_i with three learned matrices $W_i^K \in \mathbb{R}^D \otimes \mathbb{R}^{D_k}$, $W_i^V \in \mathbb{R}^D \otimes \mathbb{R}^{D_v}$ and $W_i^Q \in \mathbb{R}^D \otimes \mathbb{R}^{D_k}$, used to project keys, values and queries respectively. Each attention head applies a scaled dot-product attention (Eq. (20)) to the projected keys, values and queries:

$$h_i = \text{Attention}(KW_i^K, VW_i^V, QW_i^Q) \quad \forall i \in [1, H]. \tag{21}$$

The outputs h_i from all attention heads are concatenated into a single matrix $\in \mathbb{R}^M \otimes \mathbb{R}^{HD_v}$ and linearly reprojected onto the original D-dimensional space via an additional projection matrix $W^o \in \mathbb{R}^{HD_v} \otimes \mathbb{R}^D$.

$$\text{MHA}(K, V, Q) = \text{Concatenate}(h_1, h_2, \dots, h_H)W^o. \tag{22}$$

Transformers have an encoder–decoder structure [52]. The encoder processes the Transformer input, using self-attention to *transform* it into an hidden representation. The decoder produces the Transformer output by applying cross-attention to the hidden encoder representation and using self-attention on its previous outputs. Custom Transformers are often based on just the encoder [53] or the decoder [54], depending on the task being tackled and the adopted modelling paradigm. Transformers assume no temporal or spatial ordering of inputs. If there is any relevant input ordering for the modelling task, this must be encoded into the *input embedding*. This is commonly obtained by the addition of a *positional embedding* to the main sample embedding [52].

4. Graph Neural Networks for spatio-temporal modelling

Graph Neural Networks [55,56] model *graph-structured data*, i.e., data that can be represented using graphs. In a spatio-temporal context, graphs are quite appealing due to their suitability in representing spatial dependencies. A graph is denoted by $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V}$ and $\mathcal{E}$ are the sets of graph vertices and edges, respectively [57]. In GNNs, graphs are generally represented using two structures: a *graph signal matrix* $X \in \mathbb{R}^N \otimes \mathbb{R}^D$ and an *adjacency matrix* $A \in \mathbb{R}^N \otimes \mathbb{R}^N$, where N is the number of vertices in $\mathcal{G}$. The graph signal matrix X contains the information associated to each vertex in the graph. Each vertex has generally associated input features, represented as D-dimensional vectors and stored in the rows of X. The adjacency matrix A contains the structural information of the graph and it is a matrix whose non-zero elements A_{ij} indicate the existence of an edge in the graph connecting vertices v_i and v_j. There are some GNNs which also leverage information associated to graph edges [47,58]. In such cases, an additional structure is considered, the *edges signal matrix* $E \in \mathbb{R}^M \otimes \mathbb{R}^F$, where M is the number of graph edges. Each row in E stores an F-dimensional vector $\vec{e}_{i,j}$ carrying information on the corresponding edge between vertices v_i and v_j.

In spatio-temporal prediction, graphs are quite common when dealing with irregularly spaced spatio-temporal raster data (see Section 1). The data can be seen as a time series of graphs, where different graph signal $X(t)$ and edges signal $E(t)$ matrices are observed at each time step t. Although the adjacency matrix may change over time, the indices of its null values generally remain constant. Hence, in the graph, the connections between vertices do not change. However, the strength of connections (edge weights) can change over time. In fact, capturing dynamic connections is one of the main aspects to consider when developing spatio-temporal GNNs. Fig. 3 shows two examples of spatio-temporal graphs. In Fig. 3(a), vertex values (the graph signal matrix) change over time, as visually represented by the changing colours assigned to vertices at different time steps. The strength of their connections changes as well, as denoted by the time-changing width of each line representing an edge. In Fig. 3(b), not only vertex values and edge weights are changing, but also connections between vertices change, as some edges appear and other edges disappear between two time steps. General GNN concepts are first discussed in Section 4.1. Then, Section 4.2 discusses how GNNs are used to tackle spatio-temporal prediction tasks.

4.1. Fundamentals of graph neural networks

GNNs are based on the framework of *message passing neural networks* [47,59]. Through a multi-layer structure, GNNs *update* the feature vector of each graph vertex by *aggregating* information from neighbouring vertices, as defined by the adjacency matrix.

Consider a GNN composed of L stacked layers, the lth GNN layer computes a latent representation $\vec{h}_i^l \in \mathbb{R}^D$ of the ith vertex through a composition of $aggregate(\cdot)$ and $update(\cdot)$ operators. While processing the ith vertex, the $aggregate(\cdot)$ operator at the lth GNN layer gathers information from neighbouring vertices, as represented at the previous GNN layer, into a single vector $\vec{z}_i^{\,l}$ as follows:

$$\vec{z}_i^{\,l} = aggregate(\{\vec{h}_j^{l-1} : j \in \mathbf{N}(i)\}), \tag{23}$$

where $\mathbf{N}(i)$ is the set of those indices j such that the jth and ith vertices are neighbours in the graph. The $update(\cdot)$ operator combines the aggregated vector $\vec{z}_i^{\,l}$ with the representation $\vec{h}_i^{l-1}$ of the ith vertex at the previous GNN layer:

$$\vec{h}_i^l = update(\vec{h}_i^{l-1}, \vec{z}_i^{\,l}). \tag{24}$$

Fig. 4 gives a visual representation of the $aggregate(\cdot)$ and $update(\cdot)$ operators on an arbitrary vertex v_i, considering its three neighbours v_1, v_2 and v_3.

Input to a GNN is the original vertex information as contained in the graph signal matrix, that is, $\vec{h}_i^0 = \vec{X}_i$, where $\vec{X}_i$ indicates the ith row of the graph signal matrix. However, a common practice is to transform the original input by some parameterised transformation $f(\cdot, \theta_f)$ with learnable parameters θ_f, before feeding it to the GNN, that is, $\vec{h}_i^0 = f(\vec{X}_i, \theta_f)$.

The message passing framework is slightly different when the edge information is also considered [47]. First, an $update_{edge}(\cdot)$ operator is used to update edge features starting from the latent representations $\vec{h}_i$ and $\vec{h}_j$ of the vertices connected by the edge:

$$\vec{e}_{i,j}^{\,l} = update_{edge}(\vec{e}_{i,j}^{\,l-1}, \vec{h}_i^{\,l-1}, \vec{h}_j^{\,l-1}). \tag{25}$$

Then, while processing the ith vertex, the $aggregate(\cdot)$ operator gathers information from the feature vectors of all edges starting from v_i, namely:

$$\vec{z}_i^{\,l} = aggregate(\{\vec{e}_{i,j}^{\,l}\} : j \in \mathbf{N}(i)). \tag{26}$$

Finally, the vertex latent representation is updated as usual as indicated in Eq. (24).

A so-called *readout* layer can be included on top of the L stacked layers to use the final representations $\vec{h}_i^L$ of each vertex v_i to perform specific tasks, e.g., vertex or graph classification. In the latter case, for instance, a readout operator can be used to combine information from all vertices and compute a final vector $\vec{h}_{\mathcal{G}}$, representing the whole graph, which can then be used to classify the graph itself into one of multiple classes:

$$\vec{h}_{\mathcal{G}} = readout(\{\vec{h}_i^L : i \in [1, N]\}). \tag{27}$$

The $aggregate(\cdot)$, the $update(\cdot)$ and the $readout(\cdot)$ operators are not uniquely defined, but they should guarantee that the resulting GNN has invariance and equivariance with respect to permutation [47]. The need for both properties arises from the fact that the construction of the graph signal matrix and of the adjacency matrix requires a vertex ordering. The ordering is arbitrary and the same graph can be represented with different vertex orderings. Therefore, if the graph remains the same, vertex ordering must have no influence on GNN predictions.

According to the deep multi-layer learning paradigm, the latent vertex representations $\vec{h}_i^l$ should be more representative and useful for the task at hand the deeper the GNN is. However, an issue that may arise with GNNs is over-smoothing, where latent vertex representations become very close to each other after a certain number

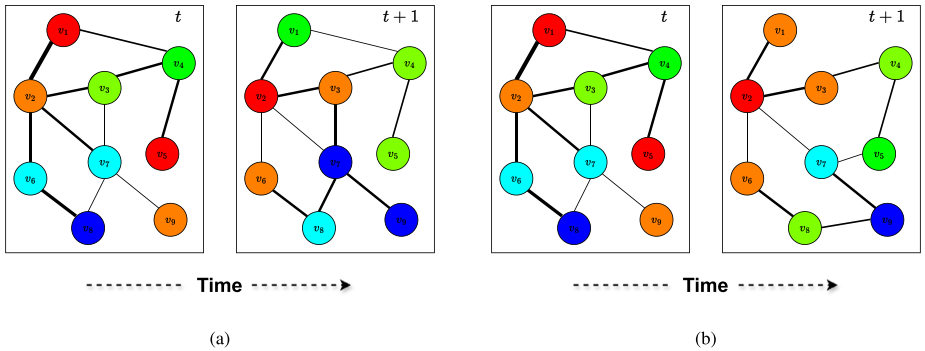


Fig. 3. Example of graphs changing over time. Colours are used to encode different vertex values. Blue corresponds to smaller values, while red corresponds to larger values. Line width is used to encode different edge weights. (3(a)) vertex values and edge weights are changing, as indicated by the different colours assigned to vertices at each time step and by the varying width of the lines representing edges, respectively. (3(b)) connections also change, as, at time $t+1$, some existing edges disappear and new edges are created.

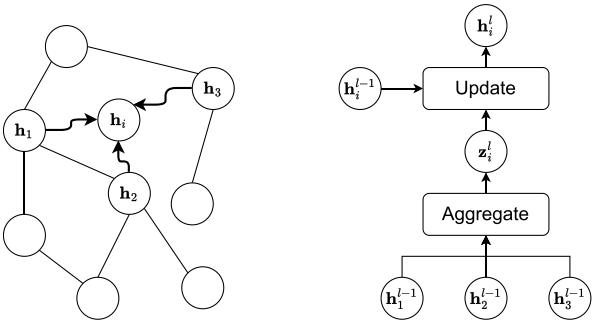


Fig. 4. Graphical representation of the $aggregate(\cdot)$ and $update(\cdot)$ operators. At a given GNN layer l, the latent representation $\vec{h}_i^l$ of an arbitrary vertex v_i is updated by aggregating the latent representations $\vec{h}_1^{l-1}$, $\vec{h}_2^{l-1}$ and $\vec{h}_3^{l-1}$ of its three neighbouring vertices.

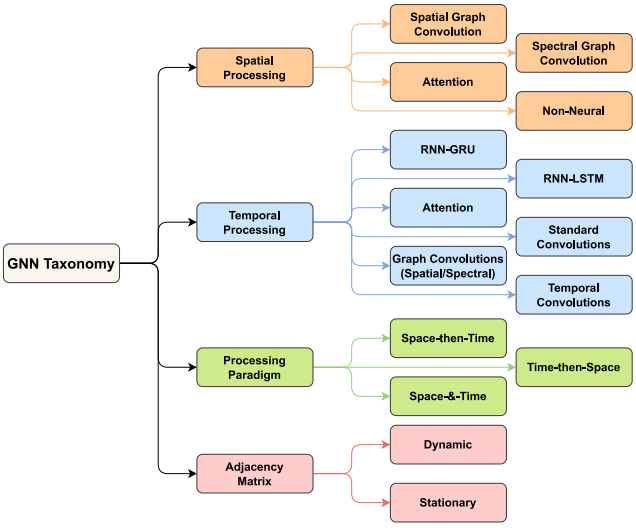


Fig. 5. An overview of the STGNNs taxonomy. Each of the four categories is indicated with different colours, along with its possible instances.

of GNN layers. This is a strong limitation since different vertices can become undistinguishable and their contribution to the final prediction becomes negligible. There are several ways to prevent over-smoothing, such as by adding residual connections for each update operation or by generating the final prediction by considering latent representations from *all* GNN layers and not just the last layer [47].

4.2. Spatio-temporal graph neural networks

In the spatio-temporal context, GNNs are employed to model spatial and temporal dependencies among entities represented as time series of spatial graphs. Many different GNN-based approaches have been proposed to model spatio-temporal dependencies, yielding diverse *Spatio-Temporal Graph Neural Networks* (STGNNs), which differ on four major aspects.

A first difference can be observed on the paradigm for the processing of the spatial and temporal domains. Some STGNNs handle space and time separately. Depending on the order in which the two dimensions are processed, they are defined as either *Time-then-Space* (*TTS*) or *Space-then-Time* (*STT*) models. Some other STGNNs, denoted as *Space & Time* (*S&T*) models,[2] process space and time jointly. A second difference lies in using different mechanisms and operations to model spatial graph dependencies. For instance, some STGNNs adopt graph convolutions while some others use attention mechanisms. Similarly, diverse STGNNs model temporal dependencies with different methodologies, such as RNNs or convolutions. Finally, STGNNs can also differ on whether they consider the adjacency matrix, and consequently the graph structure, as stationary or dynamic over time.

STGNNs can be organised in at least four categories. The first two group STGNNs according to the used approach for modelling spatial and temporal dependencies, respectively. The third category groups STGNNs based on the processing order of the spatial and temporal data. Finally, the fourth describes how different STGNNs consider the graph adjacency matrix over time, either stationary or dynamic. The proposed taxonomy is summarised in Fig. 5. Table 1 collects all the reviewed STGNN publications and categorises them according to the taxonomy groups, which will be discussed in the following sections.

4.2.1. Spatial processing

A key aspect of STGNNs is the ability to capture spatial dependencies among entities, as described by the underlying adjacency matrix. The most common approaches to model graph dependencies are based on either convolutions (see Section 3.1) or attention mechanisms (see Section 3.3). In the former, approaches are usually referred to as *Graph Convolutional Networks* (GCNs), while in the latter they are referred to as *Graph Attention Networks* (GATs). However, there are other approaches (see Table 1) that adopt peculiar methodologies that use neither convolutions nor attention.

GCNs perform the $aggregate(\cdot)$ and $update(\cdot)$ operations (see Section 4.1) using convolutions on graphs. Some works apply graph convolution in the vertex (or spatial) domain, while some others apply

[2] In this paper, the naming STT, TTS and S&T is adopted in order to be consistent with existing literature [60] that uses the same definitions.

Table 1
Taxonomy of spatio-temporal GNN publications. For each publication, the corresponding processing paradigm, the spatial and temporal processing, and the graph structure are reported.

Ref.	Spatial Processing	Temporal Processing	Processing Paradigm	Adjacency Matrix
[61]	Spatial Graph Convolution	RNN-GRU	S&T	Stationary
[62]	Spectral Graph Convolution	Causal Convolution	S&T	Stationary
[63]	Spectral Graph Convolution	RNN-LSTM	S&T	Stationary
[64]	Spatial Graph Convolution	Graph Convolution	S&T	Dynamic
[65]	Spectral Graph Convolution	Standard Convolution	STT	Dynamic
[66]	Spatial Graph Convolution	Causal Dilated Convolution	TTS	Dynamic
[20]	Spectral Graph Convolution	RNN-LSTM	TTS	Stationary
[67]	Attention	RNN-GRU	TTS	Stationary
[68]	Spectral Graph Convolution	RNN-LSTM	STT	Stationary
[69]	Spectral Graph Convolution	RNN-LSTM	STT	Stationary
[70]	Spatial Graph Convolution	Graph Convolution	S&T	Dynamic
[71]	Attention	Attention	S&T	Dynamic
[72]	Spatial Graph Convolution	Graph Convolution	S&T	Dynamic
[73]	Spectral Graph Convolution	RNN-GRU	S&T	Dynamic
[74]	Spectral Graph Convolution	Graph Convolution	S&T	Stationary
[75]	Spectral Graph Convolution	MLP	STT	Dynamic
[76]	Spectral Graph Convolution	RNN-GRU	STT	Dynamic
[77]	Spectral Graph Convolution (Graph Wavelet)	RNN-LSTM	S&T	Stationary
[78]	Spectral Graph Convolution	Causal Dilated Convolution	TTS	Stationary
[79]	Spectral Graph Convolution; Attention	Standard Convolution	TTS	Stationary
[80]	Spatial Graph Convolution	RNN-GRU	S&T	Stationary
[81]	Spectral Graph Convolution	RNN-LSTM	TTS	Dynamic
[82]	Spatial Graph Convolution; Attention	Attention	S&T	Stationary
[83]	Non-neural (ODE Solver)	Causal Dilated Convolution	S&T	Stationary
[84]	Spatial Graph Convolution	Standard Convolution	S&T	Stationary
[23]	Spectral Graph Convolution	RNN-LSTM	STT	Dynamic
[85]	Spectral Graph Convolution	RNN-LSTM	STT	Stationary
[17]	Spectral Graph Convolution	Standard Convolution	STT	Stationary
[18]	Spectral Graph Convolution	Standard Convolution	S&T	Stationary
[86]	Spectral Graph Convolution	RNN-LSTM; RNN-GRU	STT	Stationary

(*continued on next page*)

Table 1 (*continued*).

[87]	Spatial Graph Convolution	Graph Convolution	S&T	Stationary
[88]	Spatial Graph Convolution; Attention	Causal Dilated Convolution	TTS	Dynamic
[89]	Spectral Graph Convolution	Attention	TTS	Dynamic
[90]	Spectral Graph Convolution	RNN-GRU	STT	Stationary
[91]	Spectral Graph Convolution (Graph Wavelet)	Attention	STT	Dynamic
[92]	Spatial Graph Convolution	Causal Dilated Convolution	TTS	Dynamic
[65]	Attention	Attention; Standard Convolution	S&T	Stationary
[93]	Attention	RNN-GRU	STT	Dynamic
[94]	Spatial Graph Convolution	Standard Convolution; Attention; RNN-GRU	S&T	Dynamic
[95]	Spatial Graph Convolution	RNN-LSTM	STT	Stationary
[96]	Attention	RNN-LSTM; RNN-GRU	S&T	Stationary
[97]	Spectral Graph Convolution; Attention	RNN-LSTM; RNN-GRU	TTS	Stationary
[98]	Spectral Graph Convolution	RNN-LSTM	TTS	Stationary
[99]	Spatial Graph Convolution	Autoencoder	TTS	Stationary
[100]	Attention	Causal Dilated Convolution	STT	Stationary
[101]	Spatial Graph Convolution	Causal Dilated Convolution	S&T	Dynamic
[102]	Spectral Graph Convolution	RNN-LSTM	S&T	Dynamic
[103]	Spectral Graph Convolution	Standard Convolution; Attention	S&T	Stationary
[22]	Spatial Graph Convolution	RNN-GRU	STT	Stationary
[104]	Spectral Graph Convolution	RNN-GRU	STT	Dynamic
[105]	Spatial Graph Convolution	RNN-GRU	S&T	Dynamic
[19]	Spectral Graph Convolution	Graph Convolution	S&T	Stationary
[106]	Spatial Graph Convolution	Graph Convolution	S&T	Dynamic
[24]	Spatial Graph Convolution	RNN-LSTM	STT	Stationary
[21]	Spectral Graph Convolution	RNN-GRU	STT	Stationary
[107]	Spectral Graph Convolution	RNN-GRU	STT	Dynamic
[108]	Spectral Graph Convolution	RNN-GRU; Attention	STT	Dynamic
[109]	Spectral Graph Convolution	RNN-GRU; Attention	S&T	Dynamic
[110]	Spectral Graph Convolution	RNN-GRU	TTS	Stationary
[16]	Spectral Graph Convolution	Standard Convolution	TTS	Dynamic

(*continued on next page*)

Table 1 (*continued*).

[111]	Attention	Causal Dilated Convolution; Attention	TTS	Stationary
[15]	Spectral Graph Convolution; Attention	RNN-LSTM	STT	Stationary
[14]	Spectral Graph Convolution	RNN-GRU; Attention	S&T	Dynamic
[112]	Spatial Graph Convolution	Graph Convolution	S&T	Dynamic

Table 2
All reviewed STGNN publications, grouped by applicative field along with the most commonly used datasets.

Application field	References	Datasets
Traffic	[14–16,18,60–62,65–67,71–75,77–81,84,89–92,94, 96,98,102–105,107–109,111,113–115]	PeMS [116]; METR-LA [117]; NYC-Taxi [118]; NYC-Bike [119]
Environment	[18,20–22,68,69,85,86,100,101,110]	Eastern Wind Integration [120]; CCMP Wind [121]; ODIAC [122]
Trajectory	[64,70,76,82,87,99,106,112]	NTU RGB+D [123,124]; Human 3.6M [125]; Kinetics [126,127]; JAAD [128]; STIP [129]
Energy	[17–19,60,88,101]	CER-E Benchmark [130]; Solar Power Integration Data [131]
Health	[23,24,95,97]	No Clear Benchmarks

convolution in the spectral domain, i.e., on a frequency transform of the graph. Graph convolutions in the vertex domain [132,133] are direct implementations of the $aggregate(\cdot)$ and $update(\cdot)$ operators. These operators can be defined in many different ways, but they should always guarantee permutation invariance and permutation equivariance (see Section 4.1), and they should be differentiable with respect to any learnable parameter to perform gradient-based optimisation.

A very simple and popular implementation of the $aggregate(\cdot)$ operator is based on a sum of the latent representations of neighbouring vertices:

$$\vec{z}_i^{\,l} = aggregate(\{\vec{h}_j^{l-1} : j \in \mathbf{N}(i)\}) = \sum_{j \in N(i)} \vec{h}_j^{l-1}. \tag{28}$$

The $update(\cdot)$ operator is often implemented as a nonlinear function $f(\cdot)$ applied to a linear combination of the aggregated vector $\vec{z}_i^{\,l}$ and of the latent vector $\vec{h}_i^{l-1}$ of the ith vertex being considered:

$$\vec{h}_i^{l} = update(\vec{h}_i^{l-1}, \vec{z}_i^{\,l}) = f(U^l \vec{h}_i^{l-1} + W^l \vec{z}_i^{\,l} + \vec{b}^l), \tag{29}$$

where $U^l \in \mathbb{R}^D \otimes \mathbb{R}^D$, $W^l \in \mathbb{R}^D \otimes \mathbb{R}^D$ and $\vec{b}^l \in \mathbb{R}^D$ are learnable parameters of the lth GNN layer. Such implementations of the $aggregate(\cdot)$ and $update(\cdot)$ operators have equivariance with respect to permutation and a composition of multiple stacked layers implementing such operators will yield a GNN that is itself permutation equivariant.

Spectral graph convolutions rely on the *Graph Fourier Transform* and on the *convolution theorem* to perform convolutions on a graph. Consider, for simplicity, an undirected graph with N vertices and a one-dimensional graph signal (the signal matrix reduces to a vector $\vec{x} \in \mathbb{R}^N$). The Fourier transform of a graph can be formalised through the *Laplacian* $L = D - A$, where $A \in \mathbb{R}^N \otimes \mathbb{R}^N$ is the adjacency matrix and $D \in \mathbb{R}^N \otimes \mathbb{R}^N$ is the diagonal vertex degree matrix, defined such that $D_{ii} = \sum_{j=1}^{N} A_{ij}$, $i = 1, \ldots, N$. The Laplacian matrix is symmetric and positive semidefinite (consequently all its eigenvalues are non-negative). However, among the several Laplacian definitions, the *normalised Laplacian* is typically considered, namely, $L = \mathbb{I} - D^{-1/2} A D^{-1/2}$, where $\mathbb{I} \in \mathbb{R}^N \otimes \mathbb{R}^N$ is the identity matrix. The normalised Laplacian has the additional property that its largest eigenvalue is no larger than two ($\lambda_{max} <= 2$) [134]. Given its normalised Laplacian L and its graph signal $\vec{x}$, the Fourier transform of a graph is defined as the representation of its signal with respect to its Laplacian eigenvectors. In short, given the spectral decomposition of the graph Laplacian, $L = U \Lambda U^\top$, where $U \in \mathbb{R}^N \otimes \mathbb{R}^N$ is the orthogonal eigenvector matrix and $\Lambda \in \mathbb{R}^N \otimes \mathbb{R}^N$ is the corresponding diagonal eigenvalue matrix, the graph Fourier transform of the graph signal $\vec{x}$ is defined as:

$$\mathcal{F}(\vec{x}) = U^\top \vec{x}. \tag{30}$$

Since U is an orthogonal matrix ($U^{-1} = U^\top$), the inverse graph Fourier transform is given by:

$$\mathcal{F}^{-1}(\mathcal{F}(\vec{x})) = U \mathcal{F}(\vec{x}) = U U^\top \vec{x} = \vec{x}. \tag{31}$$

The *convolution theorem* can be applied, yielding:

$$\mathcal{F}(\vec{s} *_{\mathcal{G}} \vec{x}) = \mathcal{F}(\vec{s}) \odot \mathcal{F}(\vec{x}) = U^\top \vec{s} \odot U^\top \vec{x}, \tag{32}$$

where $\vec{s} *_{\mathcal{G}} \vec{x}$ denotes the graph convolution between two signals $\vec{s}$ and $\vec{x}$, based on a graph $\mathcal{G}$, and $\odot$ is the element-wise product. From Eqs. (31) and (32), spectral graph convolution can be derived:

$$\vec{y} = \vec{s} *_{\mathcal{G}} \vec{x} = \mathcal{F}^{-1}(\mathcal{F}(\vec{s} *_{\mathcal{G}} \vec{x})) = U(U^\top \vec{s} \odot U^\top \vec{x}), \tag{33}$$

where $\vec{y}$ is the *filtered* graph signal in the vertex domain.

The spectral graph representation of the signal $\vec{s}$, namely, $\vec{U}^\top \vec{s}$, can be considered as a *graph convolution kernel* $k_\theta(\Lambda)$, which is generally a function of the Laplacian eigenvalues matrix Λ and depends on some parameters θ. A *non-parametric* form of $k_\theta(\Lambda)$ can also be defined as:

$$k_\theta(\Lambda) = \mathrm{diag}(\theta) \in \mathbb{R}^N \otimes \mathbb{R}^N, \tag{34}$$

where its non-parametric nature is due to its independence on the topology of the graph, that is, on the eigenvalues Λ. Introducing the kernel $k_\theta(\Lambda)$, Eq. (33) can be rewritten as:

$$\vec{y} = k_\theta(\Lambda) *_{\mathcal{G}} \vec{x} = U k_\theta(\Lambda) U^\top \vec{x}. \tag{35}$$

The proposed formulation has two main drawbacks, limiting its applicability to practical problems. First, the computational complexity of the spectral decomposition of the graph Laplacian is $\mathcal{O}(N^3)$. Then, non-parametric spectral filters have no *localisation* in the vertex domain, that is, the effect of the filter on each graph vertex signal cannot be quantified. To overcome these limitations, two approximations are commonly used, the *Chebyshev polynomial* [135] and the *first-order* [136] approximations. Both approximations are based on the definition of spectral kernels as *Kth order polynomial filters*:

$$k_\theta(\Lambda) = \sum_{p=0}^{K} \theta_p \Lambda^p, \tag{36}$$

where K is the order of the polynomial, $\Lambda^p \in \mathbb{R}^N \otimes \mathbb{R}^N$ is the pth power of the Laplacian eigenvalues matrix and θ_p, $p \in [0, K]$ are learnable kernel parameters.

In filtering a graph signal with convolutions, as in Eq. (35), a major advantage when using polynomial filters, is the unnecessity of computing the spectral decomposition of the Laplacian matrix. By substituting the polynomial kernel definition of Eq. (36) into Eq. (35), and recalling that $U\Lambda^p U^\top = L^p$, it can be seen that graph convolution with polynomial kernels reduces to:

$$\vec{y} = k_\theta(L) *_\mathcal{G} \vec{x} = \sum_{p=0}^{K} \theta_p L^p \vec{x}, \tag{37}$$

that is, the knowledge of the graph Laplacian is adequate to perform graph convolution using polynomial kernels. Note that the kernel is a function of the graph Laplacian:

$$k_\theta(L) = \sum_{p=0}^{K} \theta_p L^p, \tag{38}$$

and the filtering operation is just a product between the kernel and the graph signal: $\vec{y} = k_\theta(L) *_\mathcal{G} \vec{x} = k_\theta(L)\vec{x}$.

The pth power of the Laplacian describes the topology of the graph. It contains information about the neighbours whose distance from a given vertex is p edges, forming the *p-hop neighbourhoods*. In fact, if $d_\mathcal{G}(i, j) > p$, then $L^p_{ij} = 0$, where $d_\mathcal{G}(i, j)$ is the number of edges on the shortest path connecting vertices i and j [137]. This property makes Eq. (37) K-localised, namely, when a given vertex is filtered, only vertices that are at maximum K steps away are considered. Consequently, spectral filters represented by Kth order polynomials of the Laplacian are K-localised.

In order to have stable and fast recursive evaluations, Deferrad et al. [135] reformulated the polynomial kernel of Eq. (38) using *pth order Chebyshev polynomials*. A Chebyshev polynomial of order p is defined recursively as $T_p(x) = T_{p-1}(x) - T_{p-2}(x)$, with $T_0(x) = 1$ and $T_1(x) = x$. In Eq. (38), Chebyshev polynomials are applied to obtain powers of the graph Laplacian. First, a scaled Laplacian is considered: $\tilde{L} = 2L/\lambda_{max} - \mathbb{I}$. Then $\tilde{L}^p$ is replaced with a pth order Chebyshev polynomial evaluated at $\tilde{L}$, $T_p(\tilde{L}) \in \mathbb{R}^N \otimes \mathbb{R}^N$, yielding a Chebyshev polynomial filter:

$$k_\theta(\tilde{L}) = \sum_{p=0}^{K} \theta_p T_p(\tilde{L}), \tag{39}$$

The filtering operation of Eq. (37) then becomes:

$$\vec{y} = k_\theta(\tilde{L}) *_\mathcal{G} \vec{x} = \sum_{p=0}^{K} \theta_p T_p(\tilde{L})\vec{x}. \tag{40}$$

Evaluating $T_p(\tilde{L})\ \forall p$, requires a single multiplication of $\tilde{L}$ by $T_{p-1}(\tilde{L})$. Since $\tilde{L}$ is a sparse matrix, the multiplication has a computational complexity of $\mathcal{O}(|\mathcal{E}|)$, where $|\mathcal{E}|$ is the edge set cardinality. Since the multiplication is performed no more than K times, the graph convolution operation using Chebyshev polynomials has a computational complexity $\mathcal{O}(K|\mathcal{E}|) \ll N^2$.

First-order approximation [136] simplifies the Chebyshev polynomial filter by considering just first-order neighbours (fixing $K = 1$ in Eq. (39)). Furthermore, it is assumed that the largest eigenvalue of the normalised Laplacian $\lambda_{max} = 2$, yielding a simple reformulation of the scaled Laplacian $\tilde{L} = L - \mathbb{I}$. Under this assumptions, and recalling the definition of the normalised Laplacian L, the filtering operation of Eq. (40) can be rewritten as:

$$\begin{aligned} \vec{y} &= \theta_0\vec{x} + \theta_1(L - \mathbb{I})\vec{x} \\ &= \theta_0\vec{x} - \theta_1 D^{-1/2} A D^{-1/2}\vec{x} \end{aligned} \tag{41}$$

Note that the filtering considers just 1-hop neighbours since it depends on the first power of the Laplacian, L^1. However, a larger receptive field can be obtained by stacking multiple layers using Eq. (41). In Eq. (41) there are two effective learnable parameters, θ_0 and θ_1, however, it can be expressed using a single learnable parameter θ:

$$\vec{y} = \theta M \vec{x}, \tag{42}$$

where $M = \mathbb{I} + D^{-1/2} A D^{-1/2}$.

Kipf and Welling [136] noted that, when stacking multiple layers implementing Eq. (42), the problems of vanishing/exploding gradients may arise. Therefore, a *renormalisation trick* has been introduced, redefining the matrix $M = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$, where $\tilde{A} = A + \mathbb{I}$ and $\tilde{D}_{ii} = \sum_{j=0}^{N} \tilde{A}_{ij}$, $i = 1, \dots, N$. Furthermore, Eq. (42) can be easily extended to account for graph signals with multiple features, that is, convolution is applied not to a vector $\vec{x} \in \mathbb{R}^N$ but to a matrix $X \in \mathbb{R}^N \otimes \mathbb{R}^D$, and to apply multiple convolutional kernels to get C-dimensional feature maps for each vertex. Graph convolution via first-order approximation is hence defined as:

$$Y = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2} X \Theta), \tag{43}$$

where, $\sigma(\cdot)$ is a nonlinear activation function, such as ReLU, $X \in \mathbb{R}^N \otimes \mathbb{R}^D$ is the multivariate graph signal matrix, $\Theta \in \mathbb{R}^D \otimes \mathbb{R}^C$ is the filter parameters matrix and $Y \in \mathbb{R}^N \otimes \mathbb{R}^C$ is the output feature map.

Computationally, first-order approximation requires less resources than Chebyshev approximation. A comparison of the performance between the approximations is carried out in [62], highlighting that the Chebyshev-based approach perform slightly better than first-order approximation. Hence, the performance loss due to first-order approximation is negligible.

Spatial and spectral graph convolutions behave differently from each other, and ultimately there is no evidence indicating which one is the best for a given specific problem. However, there is a growing interest in trying to represent both under a unifying perspective [138–140]. Spectral graph convolution captures the influence among vertices on all spectral frequencies, whereas spatial graph convolutions can be seen as low-pass filters, limiting, in this way, the information that can flow from one vertex to another. Nevertheless, spatial convolutions generally require less computational resources than spectral convolutions. In conclusion, spatial convolutions can be considered as a less expensive approximation of spectral convolutions.

Attention mechanisms [52,141] are an alternative to convolutions for modelling graph dependencies among vertices [142]. The use of attention mechanisms in GNNs is essentially motivated by the remark that the latent representation $\vec{h}_i$ of any ith vertex, can be updated as a combination of the latent representations of its neighbours, or even of all graph vertices. The combination weights are computed through an attention mechanism, weighing the importance of each neighbour's latent representation to the vertex being processed. Under a general framework, the lth layer of a GAT network updates the latent representation $\vec{h}_i^{l-1}$ of any vertex i, from the previous layer $l - 1$, as:

$$\vec{h}_i^l = \sum_{j \in \{i, N(i)\}} \alpha_{ij} \vec{h}_j^{l-1}, \tag{44}$$

where the α_{ij} are the attention weights, defined as $\alpha_{ij} = \text{softmax}_j(e_{ij})$. Moreover, unnormalised attention scores e_{ij} can be computed by applying any kind of attention, such as Bahadanau [141], multiplicative [143] or scaled dot-product [52] attentions.

Attention mechanisms on graphs can be reformulated as Multi-Head attentions, where, for each layer l, attention is applied H separate times, where H is the number of attention heads. Typically, each attention head first linearly projects the latent representations $\vec{h}_i$ of each vertex onto an M-dimensional space using a learnable projection matrix $W^k \in \mathbb{R}^M \otimes \mathbb{R}^D$, specific for the kth attention head. Attention weights α^k_{ij} are then computed using each head on the projected representations $W^k\vec{h}_i$. Then, a combination of the updated representations produced by each head $\vec{h}_i^{lk}\ \forall k \in [1, H]$, can be computed through a concatenation and reprojection layer or using gating mechanisms [144].

It is worth mentioning some works that model spatial graph dependencies using other, peculiar approaches, diverse from graph convolutions and attention mechanisms. For instance, Fang et al. [83] use a tensor-based *Ordinary Differential Equation* (ODE) solver, alongside a TCN (see Section 4.2.2), to capture jointly long-term spatial and temporal dependencies. Cini et al. [145] propose an encoder–decoder architecture, whose encoder extracts spatio-temporal features in an unsupervised manner. In particular, it uses an ESN (see Section 4.2.2) to extract temporal features for each graph vertex, and it uses a mechanism based on powers of the adjacency matrix to extract spatial features. Temporal and spatial features are then used by the decoder, based on MLPs, to predict future values for all graph vertices.

4.2.2. Temporal processing

When dealing with spatio-temporal graphs, capturing temporal dependencies among vertices is as relevant as capturing spatial dependencies. In fact, the value of a vertex at a given time is likely to depend not only on the current values of its neighbourhood, but on its own and its neighbours' past values, too. Hence, STGNNs must be able to model temporal dependencies as well. Most of the analysed STGNN models rely on RNNs (Section 3.2) and 1D convolutions (Section 3.1) to grasp temporal dependencies among vertices. As shown in Table 1, all STGNNs modelling temporal dependencies with RNNs rely on either LSTM networks [20,63,68] or GRUs [61,73,113]. Vanilla RNNs are never used and there is no clear preference towards LSTM networks or GRUs, since in literature both methods are equally popular. ESNs are also used in the context of STGNNs. For instance, a scalable and less computationally demanding STGNN leveraging a *deep ESN* for the extraction of temporal features from historical vertex data is proposed by Cini et al. [145]. The use of convolutions to model temporal dependencies is another popular choice in STGNNs. STGNNs mostly rely on 1D convolutions and TCNs, to process historical data of each vertex. Table 1 shows that, when convolution is adopted to model temporal dependencies, causal dilated convolution is preferred [66,83,88,101] over the standard one.

Another approach is to address the spatio-temporal problem in a joint *Space & Time* paradigm (see Section 4.2.3), using graph convolutions (see Section 4.2.1) to model not only spatial dependencies but temporal dependencies as well [64,70,72,106]. Another approach to temporal dependency modelling in STGNNs is based on attention mechanisms, which are becoming more and more popular in processing temporal sequences [146,147] due to their native capability in modelling long-term time dependencies. Attention mechanisms can relate two arbitrarily distant elements of a sequence directly, without requiring any further iterations, as in the RNN case, and without using many stacked layers to obtain a large receptive field, as in the case of convolutions. A STGNN leveraging attention mechanisms to model temporal dependencies is GMAN (Graph Multi Attention Network) [71], which adopts a Transformer-like, encoder–decoder structure, built around *spatio-temporal attention blocks*.

4.2.3. Processing paradigm

The previous Sections 4.2.1 and 4.2.2 describe the approaches that are mostly used in STGNNs to model spatial and temporal dependencies, respectively. These approaches are then combined together to capture spatio-temporal dependencies. This aspect is accounted for in the processing paradigm category, discussed in this section. Fundamentally, spatial and temporal processing approaches in STGNNs are combined in all, i.e., three, possible ways: *Time-then-Space* (*TTS*), *Space-then-Time* (*STT*), and *Space & Time* (*S&T*).

TTS models first process the temporal domain, extracting temporal features from each graph vertex using their peculiar temporal processing approach (e.g., GRUs or causal dilated convolutions). The extracted temporal features are then further processed using the spatial processing module, that is specific for each STGNN, along with the graph structural information provided by the adjacency matrix. The output of such two-steps processing represents graph spatio-temporal features that can be employed to perform spatio-temporal forecasting. This approach is followed by the majority of TTS models in literature [20,66,88,145]. A slightly different approach is proposed by Yu et al. [62], which inserts a graph convolution model, to extract spatial features, in between two gated convolution modules, used to extract temporal features. The same TTS methodology underlies STT models, just applied inversely: first, a spatial processing module (e.g., based on graph convolutions) is used, along with the adjacency matrix, to extract spatial features from each vertex at each time step; then, extracted spatial features are processed in time using the temporal processing approach specific to the particular STGNN.

TTS and STT models process the temporal and spatial domains sequentially, either time-first or space-first. In S&T models, the spatial and temporal domains are processed together. A common approach [61,63,73] to achieve this joint processing is to integrate graph convolutions (for spatial dependency modelling) directly into an LSTM or GRU (modelling temporal dependencies). For instance, Li et al. [61] replace the canonical matrix multiplications within the GRU's reset and update gates (see Eq. (14)) with graph convolutions. Therefore, equations defining the reset and update gates, in this case, become:

$$\vec{r}_t = \sigma\left(k_{\theta_r} *_{\mathcal{G}} [X_t, H_{t-1}] + \vec{b}^r\right), \tag{45}$$

$$\vec{u}_t = \sigma\left(k_{\theta_u} *_{\mathcal{G}} [X_t, H_{t-1}] + \vec{b}^u\right), \tag{46}$$

where, k_{θ_r} and k_{θ_u} are the graph convolution kernels, with parameters θ_r and θ_u for the reset and update gates, respectively; $[\cdot,\cdot]$ is a concatenation operator; $*_{\mathcal{G}}$ is here used to indicate a generic graph convolution, such as, graph convolution based on first order approximation (Eq. (43)). Similarly, graph convolution is integrated in Eq. (15), used to compute the intermediate output, as follows:

$$\vec{h}'_t = \tanh\left(k_\theta *_{\mathcal{G}} [X_t, (\vec{r}_t \odot H_{t-1})] + \vec{b}\right), \tag{47}$$

Alternative ways to achieve a joint spatio-temporal processing have been investigated. For instance, Zheng et al. [71] propose GMAN, a full-attention Transformer-like architecture that processes space and time in parallel using attention mechanisms. Then, spatio-temporal features are produced by merging together, through a gating mechanism, the features extracted using the spatial and temporal attention. Song et al. [72] propose *Spatial-Temporal Synchronous Graph Convolutions* (STSGCs), that jointly capture spatio-temporal dependencies by applying graph convolution on extended adjacency matrices. An extended adjacency matrix is constructed for each time step, and in any given matrix, each vertex is connected not only to its neighbourhood at the current time step, as in the canonical adjacency matrix, but it is connected to itself at the preceding and next time steps, too. Graph convolution is applied separately on each extended adjacency matrix, yielding localised spatio-temporal features for each time step. Akin to standard convolutional layers, multiple stacked STSGC layers allow the extraction of high level spatio-temporal features with an increasing receptive field. A similar approach is adopted in [64,70,106], where spatial graph convolution is applied on an extended space–time adjacency matrix, thus extracting not only spatial but also temporal dependencies. These matrices are built as in [72], by connecting each vertex also to itself but at adjacent time steps; however each graph convolution has a larger kernel size over the temporal dimension and thus a larger temporal receptive field. Finally, Fang et al. [83] jointly model spatio-temporal features through a tensor-based Ordinary Differential Equation (ODE) solver, coadiuvated by two temporal convolution modules.

None of these processing paradigms clearly outperforms the others, also because there are further factors influencing STGNNs' performance. However, S&T modelling should be preferred since it allows modelling spatial and temporal dependencies jointly; as they naturally appear in data, and not sequentially, which might break some of data's intrinsic spatio-temporal dependencies.

Instead of proposing a new STGNN model, Wang et al. [115] introduces a Neural Architecture Search (NAS) approach, based on *reinforcement learning*, to automatically determine the optimal combination of spatial and temporal processing approaches for a given task. It leverages spatio-temporal processing ideas proposed in [62,65,72] and automatically combines them together to obtain an empirically optimal STGNN model using reinforcement learning. It is also worth mentioning that in Cini et al. [60] the attention is focused on how to capture local vertex dynamics using STGNNs. The idea is that besides global dynamics, each vertex can show local and specific dynamics over time which should be accounted for by STGNNs. Their approach is based on *node embeddings*, that capture local dynamics of each individual vertex in the graph. The STGNN model is then applied to this type of embedding, which captures and, in a certain sense, abstracts the model from the local dynamics of the individual vertices. This approach is empirically shown to be a good middle ground between models that explicitly consider locality (e.g., using many specific models, one for each vertex) but fail to exploit relationships and dependencies among diverse vertices, and global models which do the exact opposite, trying to capture global relations, but failing in accounting for local, vertex-specific dynamics.

4.2.4. Graph representation

In temporally evolving graph tasks, it is required to grasp the dynamism of the graph structure, namely, of the adjacency matrix's dynamism. In fact, not only vertex values change over time, but graph edges may change as well. When mining spatio-temporal dependencies on graphs, changes in graph edges are as relevant as those in the graph signal matrix, since they imply that each vertex exerts an influence on the others that changes over time. Three main approaches concerning the dynamism in spatio-temporal graphs can be identified.

First, there are approaches that consider the adjacency matrix, and thus the relationships among vertices, stationary and constant in time. The adjacency matrix is fixed by leveraging prior knowledge about the graph structure. For example, in many traffic forecasting problems, the adjacency matrix is built to reflect the physical distance between vertices [61,63,113,145] and sometimes, when available, even the road network structure [62]. These approaches do not explicitly account for possible changes in graph edges and they apply their own peculiar spatio-temporal processing approach to the pre-defined and fixed adjacency matrix.

Secondly, there are approaches that explicitly model graph dynamism by dealing with vertex relationships differently in time. Some works [65,71,89] use attention mechanisms in space and time to dynamically adjust the strength of the connections between different vertices and different time steps, respectively, according to their values. A similar idea is followed by [64,70], which use learned masks to weigh the influence that each vertex exerts on the others. A particular approach is used by Song et al. [72]. In spite of utilising a static adjacency matrix, it deploys multiple graph convolution blocks, each focusing on a specific time step with its own parameters, so that each block can learn vertex dynamics at its specific time step. Another approach to model graph dynamism is based on *meta-learning* [93]. It uses the information associated with vertices and edges, at each time step, to get, through simple neural networks, the weights of the modules used for the spatial and temporal processing. Since their parameters depend on time-specific attributes of vertices and edges, spatial and temporal processing modules adapt automatically and indirectly themselves to dynamic relationships in the graph structure.

Half-way in between the aforementioned two approaches, there are those where the adjacency matrix is learned directly from data (*graph learning*) [66,73,88,101]. This learned matrix is still stationary but it is constructed directly from data during training and it can better catch relationships among vertices that are unlikely to be captured by a-priori fixed adjacency matrices. This approach is also useful in those cases where the adjacency matrix is latent and cannot be computed directly, such as in any multivariate time-series forecasting problems [101]. Furthermore, a learned adjacency matrix can also complement an a-priori known adjacency matrix to compensate for unknown information that is potentially missing from the latter [66].

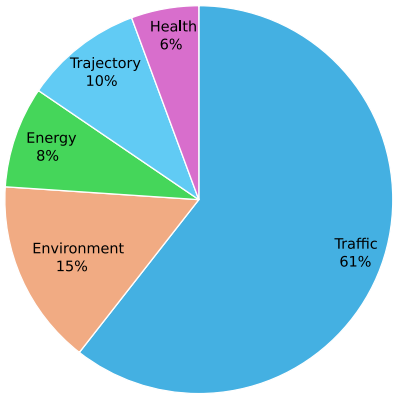


Fig. 6. Pie chart depicting the distribution of analysed STGNNs articles in the most common applicative fields. The vast majority of STGNNs applications fall into the traffic field. Other major applicative fields, reported by descending popularity, are environment, trajectory prediction, energy and health.

4.3. Applications, datasets and metrics

The section provides a literature analysis from an application-oriented perspective. Three main areas will be discussed: the applicative fields in which STGNNs are most commonly used; the most widely adopted datasets for each of the aforementioned fields; the metrics that are most commonly used to evaluate and compare different STGNNs architectures. Table 2 summarises all reviewed STGNN articles from an applicative perspective. Reviewed STGNN articles cover a variety of applicative fields. This work identifies five main areas: *traffic forecasting*, *environmental prediction*, *trajectory modelling*, *energy* and *health*. Fig. 6 gives an overview on the distribution of articles in each of the aforementioned fields.

The field in which STGNNs are most commonly adopted is, by far, *traffic forecasting* [61,62,65,72,113], where the goal is to predict future traffic conditions over a network of road traffic sensors starting from past traffic conditions. The evolution of traffic conditions over a road network can be modelled as a time series of graphs and thus STGNNs are particularly suitable to solve traffic forecasting problems. In these tasks, graph vertices represent traffic sensors located on roads and junctions and edges encode the connections between sensors, reflecting the underlying road network. The most popular dataset in this domain is *Caltrans' PeMS* (*Performance Measurement System*) [116], which has become a *de facto* benchmark. The dataset collects data from thousands of detectors located along the freeway system connecting all major metropolitan areas of California. Another dataset used for traffic forecasting with STGNNs is *METR-LA* [117], containing traffic information collected from loop detectors in the highway of the Los Angeles County. The commonly used METR-LA data focuses on traffic information from March to June 2012. Other minor datasets for traffic forecasting are the *New York City Taxi* (NYCTaxi) [118] and *New York City Bike* (NYCBike) [119].

There is a widespread adoption of STGNNs in the *environmental domain*, too. This field covers heterogeneous works dealing with environmental related tasks, e.g., wind speed prediction [20,101], pollutant concentration prediction [68,86,110], air quality prediction [22,85] and more. In these tasks, measuring sensors, such as wind farms [20], monitoring stations [22,68], parking lots [69], are usually dislocated irregularly over a predefined spatial area. Each sensor acquires measurements over time, thus yielding a spatial time series. The sensors are

naturally modelled as vertices in a graph. Since the sensors' connectivity might not be available, edges between vertices are typically defined according to the similarity between time series. Therefore, two sensors are connected by an edge if their respective time series are similar enough. The edge weight generally depends on the spatial distance between vertices. In these fields there are no clear benchmarks and custom private datasets are usually adopted. Nevertheless, some relevant datasets can be identified. Two datasets concerning spatio-temporal wind speed modelling are the *Eastern Wind Integration Dataset* [120] and the *Cross-Calibrated Multi-Platform* (CCMP) wind dataset [121]. A CO2 emissions dataset often used is the *Open-Data Inventory for Anthropogenic Carbon dioxide* (ODIAC) dataset [122]. Air quality and air pollution data provided by governmental organisations and by the World Air Quality Index project are often used.

Another field in which STGNNs are being used is *trajectory modelling* and *prediction*. Tasks commonly tackled within this field are pedestrian trajectory prediction [76,82,87], object tracking [112] and skeleton-based human action recognition [64,70,106]. The former consists in predicting future positions of a given number of pedestrians within a scene starting from their past positions. In this case, graph construction is not trivial. It is usually performed on the output of a semantic segmentation of video frames [76]; then, for each frame a graph is constructed, whose vertices are semantic instances within the frame. Each pedestrian instance is then connected to all instances with which it might interact. Furthermore, pedestrian instances are connected in time to capture the temporal evolution of their trajectory. The latter case is about identifying the action being performed by one or more actors in a scene starting from motion data. Unlike image-based action recognition, which tries to identify human action starting from RGB videos, skeleton-based action recognition is naturally modelled using a temporal sequence of graphs. These graphs capture the human skeleton structure, such as, that the elbow is attached to the shoulder and to the wrist, and so on. Moreover, they also grasp higher level semantic connections, such that the hand and the foot have to interact in an action such as 'tying shoes'. Typically, the former connections are statically encoded, while the latter are learned and discovered during training [106]. For skeleton-based human action recognition, the *NTU RGB+D* dataset [123,124] and the *Human 3.6M* [125] emerge as popular benchmarks. They are large scale datasets providing annotated examples of human actions in both RGB and skeleton-based data formats. Another relevant benchmark is the *Kinetics* dataset [126,127] from DeepMind. It is a collection of videos and, unlike the former two datasets, it does not natively provide skeleton-based data. A common approach is to extract such data format [64,70] by using pose estimation models such as OpenPose [148]. For trajectory prediction there are no clearly established benchmarks, however, two used datasets are the *Joint Attention in Autonomous Driving* (JAAD) [128] and the *Stanford-TRI Intent Prediction* (STIP) [129] datasets.

Another minor applicative area is *energy*, where STGNNs are used for electric load forecasting [88,101] and solar power plants output prediction [18,19,101] using, e.g., the *CER-E Benchmark* [130] and the *Solar Power Integration Data* [131], respectively. A further area is *health*, where STGNNs have been used to predict mostly Covid-19 spread data [23,24,97] and spatio-temporal EEG signals [95]. Finally, it is worthwhile to remark how STGNNs can be used to model video prediction tasks and language modelling tasks [63] and generic multivariate time series forecasting tasks, in which no adjacency matrix is known a-priori [101].

When comparing different STGNNs architectures, it is important to adopt the same *evaluation metrics* used in literature. After analysing published STGNNs articles on predictive tasks, what emerges is that three are, by a great margin, the most used metrics: the *mean absolute error* (MAE), the *root mean square error* (RMSE) and the *mean absolute percentage error* (MAPE). Given a collection of N ground truth observations $\{y_i\}_{i=1}^N$ and the corresponding predicted values $\{\hat{y}_i\}_{i=1}^N$, MAE, described in Eq. (48), is defined as the average difference, in absolute value, between a ground truth observation y_i and the corresponding predicted value $\hat{y}_i$.

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{48}$$

RMSE, described in Eq. (49), is given by the square root of the average squared difference between a ground truth observation y_i and the corresponding predicted value $\hat{y}_i$.

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{49}$$

Both the MAE and RMSE are errors expressed at the same scale and in the same unit of the observations y_i, therefore, when using them to compare different models, it is important that they work with values having the same scale and the same unit, such as by training them on the same dataset. Unlike MAE and RMSE, MAPE expresses the error in percentage units instead of the observation's units. Fundamentally, it is a scaled version of the MAE, in which each error is scaled and made relative to the ground truth observation y_i. MAPE is defined in Eq. (50).

$$MAPE = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{50}$$

These three metrics are commonly used in predictive tasks such as forecasting, where the goal is to predict a value that has the same scale and is in the same units as the ground truth data, since they are particularly suited to measure *reconstruction errors*. Furthermore, MAE and RMSE are often used directly as loss functions as well, to train the predictive STGNN models, with the MAE being used more frequently as loss than the RMSE.

5. STGNN limitations and open problems

Some major problems in STGNNs remain open. The first one resides in the inability of GNNs to estimate an uncertainty measure for spatio-temporal prediction using the aforementioned processing approaches. In principle, all STGNNs models described in this work can be trained using *Bayesian learning* [149] to get reliability measures of the model prediction. For instance, this can be achieved by using *Flow Matching* [150] or *Diffusion Models* [151,152] based on GNNs [153–155].

The second problem concerns the development of very large STGNN models. Such architectures require a huge quantity of training data. Such data are not always available, and even if they are, their processing often requires an intractable computational burden. This limitation hinders STGNNs' scalability and deployment over large scale spatio-temporal graphs. An approach to improve STGNNs' scalability is *graph sampling* [156–158]. Furthermore, in large and deep STGNN models, the over-smoothing problem (see Section 4.1) may occur, mining the model performance.

Moreover, a further limitation concerns the limited *explainability* of STGNN models, since they do not provide clear and comprehensive motivations on the given prediction, as they resemble black-box models. However, the use of certain methodologies, such as, attention-based models, seems to address partially the aforementioned limitation. Furthermore, a tool that could be integrated into STGNNs to enhance their explainability is *GNNExplainer* [159], providing explanations on GNN outputs.

A further relevant issue in STGNNs is the *immediate* information propagation between nodes, while in real world scenarios there is an information propagation delay. To properly capture spatio-temporal dependencies, such delays have to be modelled. To this purpose, some works cope with the issue by integrating GNNs with partial differential equations [160].

Finally, another problem in spatio-temporal prediction remains the presence *of missing data*, namely, the presence of spatio-temporal samples whose values are not known. The existence of missing data is quite common in both regularly and irregularly spaced spatio-temporal data and it is an issue that cannot be neglected. For instance, the simple deletion of missing data cannot be applied since the spatio-temporal dependencies would be destroyed. In the irregularly-spaced case, some attempts to cope with this problem using STGNNs has been performed [149], even if a full and convincing solution to the problem remains to be proposed.

6. Conclusions

Graph Neural Networks for spatio-temporal prediction herein have been surveyed, providing a taxonomy of the STGNN architectures proposed in literature. Moreover, a detailed description of the core STGNN components is given, focusing on spatial and temporal processing modules, the processing paradigm and graph representation. A comprehensive search has been conducted in the refereed literature, highlighting the most popular benchmarks and metrics in diverse applicative domains. In addition, a description of statistical methods for spatio-temporal prediction is provided. Finally, a proper section has been devoted to the discussion of the main drawbacks of STGNN architectures.

Having said that, some problems remain open. For instance, STGNN models, based on the deep learning approaches covered in this work, do not provide uncertainty measures for their prediction. Although there are a few attempts in literature, future developments will concern a comprehensive investigation of GNN architectures trained by means of Bayesian learning, including Flow Matching and Diffusion Models. Furthermore, given the importance of deep learning interpretability in certain domains, e.g., medicine, environment, additional efforts should be made in improving the interpretability of STGNN models.

CRediT authorship contribution statement

Vincenzo Capone: Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **Angelo Casolaro:** Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **Francesco Camastra:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] C. Xiao, N. Chen, C. Hu, K. Wang, Z. Xu, Y. Cai, L. Xu, Z. Chen, J. Gong, A spatiotemporal deep learning model for sea surface temperature field prediction using time-series satellite data, Environ. Model. Softw. 120 (2019) 104502, http://dx.doi.org/10.1016/j.envsoft.2019.104502.

[2] H. Zhou, F. Zhang, Z. Du, R. Liu, Forecasting PM2.5 using hybrid graph convolution-based model considering dynamic wind-field to offer the benefit of spatial interpretability, Environ. Pollut. 273 (2021) 116473, http://dx.doi.org/10.1016/j.envpol.2021.116473.

[3] A.M. Censi, D. Ienco, Y.J.E. Gbodjo, R.G. Pensa, R. Interdonato, R. Gaetano, Attentive spatial temporal graph CNN for land cover mapping from multi temporal remote sensing data, IEEE Access 9 (2021) 23070–23082, http://dx.doi.org/10.1109/ACCESS.2021.3055554.

[4] Y. Matsubara, Y. Sakurai, W.G. Van Panhuis, C. Faloutsos, FUNNEL: automatic mining of spatially coevolving epidemics, in: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, New York New York USA, 2014, pp. 105–114, http://dx.doi.org/10.1145/2623330.2623624.

[5] K. Glomb, J. Rué Queralt, D. Pascucci, M. Defferrard, S. Tourbier, M. Carboni, M. Rubega, S. Vulliémoz, G. Plomp, P. Hagmann, Connectome spectral analysis to track EEG task dynamics on a subsecond scale, NeuroImage 221 (2020) 117137, http://dx.doi.org/10.1016/j.neuroimage.2020.117137.

[6] T. Azevedo, A. Campbell, R. Romero-Garcia, L. Passamonti, R.A. Bethlehem, P. Liò, N. Toschi, A deep graph neural network architecture for modelling spatio-temporal dynamics in resting-state functional MRI data, Med. Image Anal. 79 (2022) 102471, http://dx.doi.org/10.1016/j.media.2022.102471.

[7] K.G. Pillai, R.A. Angryk, J.M. Banda, M.A. Schuh, T. Wylie, Spatio-temporal co-occurrence pattern mining in data sets with evolving regions, in: 2012 IEEE 12th International Conference on Data Mining Workshops, IEEE, Brussels, Belgium, 2012, pp. 805–812, http://dx.doi.org/10.1109/ICDMW.2012.130.

[8] B. Aydin, D. Kempton, V. Akkineni, R. Angryk, K. Pillai, Mining spatiotemporal co-occurrence patterns in solar datasets, Astron. Comput. 13 (2015) 136–144, http://dx.doi.org/10.1016/j.ascom.2015.10.003.

[9] R. Morosin, J. De La Cruz Rodríguez, C.J. Díaz Baso, J. Leenaarts, Spatio-temporal analysis of chromospheric heating in a plage region, Astron. Astrophys. 664 (2022) A8, http://dx.doi.org/10.1051/0004-6361/202243461.

[10] N.A.C. Cressie, Statistics for Spatial Data, revised ed., Wiley Series in Probability and Mathematical Statistics, Wiley, New York, 1993, http://dx.doi.org/10.1002/9781119115151.

[11] N.A.C. Cressie, C.K. Wikle, Statistics for Spatio-Temporal Data, in: Wiley Series in Probability and Statistics, Wiley, Hoboken, N.J, 2011.

[12] G. Atluri, A. Karpatne, V. Kumar, Spatio-temporal data mining: A survey of problems and methods, ACM Comput. Surv. 51 (4) (2019) 1–41, http://dx.doi.org/10.1145/3161602.

[13] S. Wang, J. Cao, P.S. Yu, Deep learning for spatio-temporal data mining: A survey, IEEE Trans. Knowl. Data Eng. 34 (8) (2020) 3681–3700, http://dx.doi.org/10.1109/TKDE.2020.3025580.

[14] Y. Ma, H. Lou, M. Yan, F. Sun, G. Li, Spatio-temporal fusion graph convolutional network for traffic flow forecasting, Inf. Fusion 104 (2024) 102196, http://dx.doi.org/10.1016/j.inffus.2023.102196.

[15] Y. Zhang, S. Xu, L. Zhang, W. Jiang, S. Alam, D. Xue, Short-term multi-step-ahead sector-based traffic flow prediction based on the attention-enhanced graph convolutional LSTM network (AGC-LSTM), Neural Comput. Appl. (2024) http://dx.doi.org/10.1007/s00521-024-09827-3.

[16] K. Guo, D. Tian, Y. Hu, Y. Sun, Z.S. Qian, J. Zhou, J. Gao, B. Yin, Contrastive optimized graph convolution network for traffic forecasting, Neurocomputing 602 (2024) 128249, http://dx.doi.org/10.1016/j.neucom.2024.128249.

[17] Y. Luo, C. Lu, L. Zhu, J. Song, Data-driven short-term voltage stability assessment based on spatial-temporal graph convolutional network, Int. J. Electr. Power Energy Syst. 130 (2021) 106753, http://dx.doi.org/10.1016/j.ijepes.2020.106753.

[18] Y. Wu, D. Zhuang, A. Labbe, L. Sun, Inductive graph neural networks for spatiotemporal kriging, in: Proceedings of the AAAI Conference on Artificial Intelligence, 35, (5) 2021, pp. 4478–4485, http://dx.doi.org/10.1609/aaai.v35i5.16575.

[19] M. Zhang, Z. Zhen, N. Liu, H. Zhao, Y. Sun, C. Feng, F. Wang, Optimal graph structure based short-term solar PV power forecasting method considering surrounding spatio-temporal correlations, IEEE Trans. Ind. Appl. 59 (1) (2023) 345–357, http://dx.doi.org/10.1109/TIA.2022.3213008.

[20] M. Khodayar, J. Wang, Spatio-temporal graph deep neural network for short-term wind speed forecasting, IEEE Trans. Sustain. Energy 10 (2) (2019) 670–681, http://dx.doi.org/10.1109/TSTE.2018.2844102.

[21] I.-F. Su, Y.-C. Chung, C. Lee, P.-M. Huang, Effective PM2.5 concentration forecasting based on multiple spatial–temporal GNN for areas without monitoring stations, Expert Syst. Appl. 234 (2023) 121074, http://dx.doi.org/10.1016/j.eswa.2023.121074.

[22] X.-B. Jin, Z.-Y. Wang, J.-L. Kong, Y.-T. Bai, T.-L. Su, H.-J. Ma, P. Chakrabarti, Deep spatio-temporal graph network with self-optimization for air quality prediction, Entropy 25 (2) (2023) 247, http://dx.doi.org/10.3390/e25020247.

[23] V. La Gatta, V. Moscato, M. Postiglione, G. Sperli, An epidemiological neural network exploiting dynamic graph structured data applied to the COVID-19 outbreak, IEEE Trans. Big Data 7 (1) (2021) 45–55, http://dx.doi.org/10.1109/TBDATA.2020.3032755.

[24] K. Skianis, G. Nikolentzos, B. Gallix, R. Thiebaut, G. Exarchakis, Predicting COVID-19 positivity and hospitalization with multi-scale graph neural networks, Sci. Rep. 13 (1) (2023) 5235, http://dx.doi.org/10.1038/s41598-023-31222-6.

[25] P. Wu, Z. Yin, H. Yang, Y. Wu, X. Ma, Reconstructing geostationary satellite land surface temperature imagery based on a multiscale feature connected convolutional neural network, Remote. Sens. 11 (3) (2019) 300, http://dx.doi.org/10.3390/rs11030300.

[26] T.R. Andersson, J.S. Hosking, M. Pérez-Ortiz, B. Paige, A. Elliott, C. Russell, S. Law, D.C. Jones, J. Wilkinson, T. Phillips, J. Byrne, S. Tietsche, B.B. Sarojini, E. Blanchard-Wrigglesworth, Y. Aksenov, R. Downie, E. Shuckburgh, Seasonal Arctic sea ice forecasting with probabilistic deep learning, Nat. Commun. 12 (1) (2021) 5124, http://dx.doi.org/10.1038/s41467-021-25257-4.

[27] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-k. Wong, W.-c. Woo, Convolutional LSTM network: A machine learning approach for precipitation nowcasting, in: C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 28, Curran Associates, Inc., 2015, pp. 802–810, URL: https://papers.nips.cc/paper_files/paper/2015/hash/07563a3fe3bbe7e3ba84431ad9d055af-Abstract.html.

[28] X. Shi, Z. Gao, L. Lausen, H. Wang, D.-Y. Yeung, W.-k. Wong, W.-c. WOO, Deep learning for precipitation nowcasting: A benchmark and a new model, in: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017, pp. 5622–5632, URL: https://papers.nips.cc/paper_files/paper/2017/hash/a6db4ed04f1621a119799fd3d7545d3d-Abstract.html.

[29] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: Transformers for image recognition at scale, 2020, http://dx.doi.org/10.48550/ARXIV.2010.11929.

[30] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, C. Schmid, ViViT: A video vision transformer, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, ICCV, 2021, pp. 6836–6846, http://dx.doi.org/10.1109/ICCV48922.2021.00676.

[31] Z. Liu, J. Ning, Y. Cao, Y. Wei, Z. Zhang, S. Lin, H. Hu, Video swin transformer, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2022, pp. 3202–3211, http://dx.doi.org/10.1109/CVPR52688.2022.00320.

[32] Z. Gao, X. Shi, H. Wang, Y. Zhu, Y.B. Wang, M. Li, D.-Y. Yeung, Earthformer: Exploring space-time transformers for earth system forecasting, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), in: Advances in Neural Information Processing Systems, vol. 35, Curran Associates, Inc., 2022, pp. 25390–25403, URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/a2affd71d15e8fedffe18d0219f4837a-Abstract-Conference.html.

[33] K.-H.N. Bui, J. Cho, H. Yi, Spatial-temporal graph neural network for traffic forecasting: An overview and open research issues, Appl. Intell. 52 (3) (2022) 2763–2774, http://dx.doi.org/10.1007/s10489-021-02587-w.

[34] W. Jiang, J. Luo, Graph neural network for traffic forecasting: A survey, Expert Syst. Appl. 207 (2022) 117921, http://dx.doi.org/10.1016/j.eswa.2022.117921.

[35] G. Jin, Y. Liang, Y. Fang, Z. Shao, J. Huang, J. Zhang, Y. Zheng, Spatio-temporal graph neural networks for predictive learning in urban computing: A survey, IEEE Trans. Knowl. Data Eng. 36 (10) (2024) 5388–5408, http://dx.doi.org/10.1109/TKDE.2023.3333824.

[36] Y. Wang, Advances in spatiotemporal graph neural network prediction research, Int. J. Digit. Earth 16 (1) (2023) 2034–2066, http://dx.doi.org/10.1080/17538947.2023.2220610.

[37] C.K. Wikle, A. Zammit Mangion, N.A.C. Cressie, Spatio-Temporal Statistics with R, Chapman & Hall/CRC: The R Series, CRC Press Taylor & Francis Group, Boca Raton London New York, 2019, URL: https://spacetimewithr.org/book.

[38] C.K. Wikle, A. Zammit-Mangion, Statistical deep learning for spatial and spatiotemporal data, Annu. Rev. Stat. Appl. 10 (1) (2023) 247–270, http://dx.doi.org/10.1146/annurev-statistics-033021-112628.

[39] G. Box, G. Jenkins, Time Series Analysis: Forecasting and Control, in: Holden-Day Series in Time Series Analysis and Digital Processing, Holden-Day, 1976.

[40] P.E. Pfeifer, S.J. Deutsch, A three-stage iterative procedure for space-time modeling, Technometrics 22 (1) (1980) 35, http://dx.doi.org/10.2307/1268381.

[41] D.S. Stoffer, Estimation and identification of space-time ARMAX models in the presence of missing data, J. Amer. Statist. Assoc. 81 (395) (1986) 762–772, http://dx.doi.org/10.1080/01621459.1986.10478333.

[42] P.E. Pfeifer, S.J. Deutsch, Seasonal space-time ARIMA modeling, Geogr. Anal. 13 (2) (1981) 117–133, http://dx.doi.org/10.1111/j.1538-4632.1981.tb00720.x.

[43] C.K. Wikle, M.B. Hooten, A general science-based framework for dynamical spatio-temporal models, TEST 19 (3) (2010) 417–451, http://dx.doi.org/10.1007/s11749-010-0209-z.

[44] C.K. Wikle, Comparison of deep neural networks and deep hierarchical models for spatio-temporal data, J. Agric. Biol. Environ. Stat. 24 (2) (2019) 175–203, http://dx.doi.org/10.1007/s13253-019-00361-7.

[45] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016, URL: http://www.deeplearningbook.org.

[46] C. Lea, M.D. Flynn, R. Vidal, A. Reiter, G.D. Hager, Temporal convolutional networks for action segmentation and detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR, 2017, pp. 1003–1012, http://dx.doi.org/10.1109/CVPR.2017.113.

[47] C.M. Bishop, H. Bishop, Deep Learning: Foundations and Concepts, first ed., Springer International Publishing, 2024, http://dx.doi.org/10.1007/978-3-031-45468-4.

[48] J.L. Elman, Finding structure in time, Cogn. Sci. 14 (2) (1990) 179–211, http://dx.doi.org/10.1207/s15516709cog1402_1.

[49] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780, http://dx.doi.org/10.1162/neco.1997.9.8.1735.

[50] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using RNN encoder-decoder for statistical machine translation, 2014, http://dx.doi.org/10.48550/ARXIV.1406.1078, Version Number: 3.

[51] H. Jaeger, The "Echo State" Approach to Analysing and Training Recurrent Neural Networks-With an Erratum Note, Technical Report 148, German National Research Center for Information Technology GMD, Bonn, Germany, 2001.

[52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017, pp. 6000–6010, URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[53] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, 2018, http://dx.doi.org/10.48550/ARXIV.1810.04805.

[54] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, Improving language understanding by generative pre-training, 2018.

[55] M. Gori, G. Monfardini, F. Scarselli, A new model for learning in graph domains, in: Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005, vol. 2, IEEE, Montreal, Que., Canada, 2005, pp. 729–734, http://dx.doi.org/10.1109/IJCNN.2005.1555942.

[56] F. Scarselli, M. Gori, Ah Chung Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, IEEE Trans. Neural Netw. 20 (1) (2009) 61–80, http://dx.doi.org/10.1109/TNN.2008.2005605.

[57] T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, Introduction to Algorithms, Fourth ed., The MIT Press, Cambridge, Massachusetts London, England, 2022, URL: https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/.

[58] P.W. Battaglia, J.B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, R. Pascanu, Relational inductive biases, deep learning, and graph networks, 2018, http://dx.doi.org/10.48550/ARXIV.1806.01261.

[59] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: D. Precup, Y.W. Teh (Eds.), Proceedings of the 34th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 70, PMLR, 2017, pp. 1263–1272, URL: https://proceedings.mlr.press/v70/gilmer17a.html.

[60] A. Cini, I. Marisca, D. Zambon, C. Alippi, Taming local effects in graph-based spatiotemporal forecasting, in: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), in: Advances in Neural Information Processing Systems, vol. 36, Curran Associates, Inc., 2023, pp. 55375–55393, URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/ad58c61c71efd5436134a3ecc87da6ea-Paper-Conference.pdf.

[61] Y. Li, R. Yu, C. Shahabi, Y. Liu, Diffusion convolutional recurrent neural network: Data-driven traffic forecasting, in: International Conference on Learning Representations, 2018, URL: https://openreview.net/forum?id=SJiHXGWAZ.

[62] B. Yu, H. Yin, Z. Zhu, Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence Organization, Stockholm, Sweden, 2018, pp. 3634–3640, http://dx.doi.org/10.24963/ijcai.2018/505.

[63] Y. Seo, M. Defferrard, P. Vandergheynst, X. Bresson, Structured sequence modeling with graph convolutional recurrent networks, in: L. Cheng, A.C.S. Leung, S. Ozawa (Eds.), in: Neural Information Processing, vol. 11301, Springer International Publishing, Cham, 2018, pp. 362–373, URL: https://link.springer.com/10.1007/978-3-030-04167-0_33.

[64] S. Yan, Y. Xiong, D. Lin, Spatial temporal graph convolutional networks for skeleton-based action recognition, in: Proceedings of the AAAI Conference on Artificial Intelligence, 32, (1) 2018, http://dx.doi.org/10.1609/aaai.v32i1.12328.

[65] S. Guo, Y. Lin, N. Feng, C. Song, H. Wan, Attention based spatial-temporal graph convolutional networks for traffic flow forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, 33, (01) 2019, pp. 922–929, http://dx.doi.org/10.1609/aaai.v33i01.3301922.

[66] Z. Wu, S. Pan, G. Long, J. Jiang, C. Zhang, Graph WaveNet for deep spatial-temporal graph modeling, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence Organization, Macao, China, 2019, pp. 1907–1913, http://dx.doi.org/10.24963/ijcai.2019/264.

[67] Z. Pan, Y. Liang, W. Wang, Y. Yu, Y. Zheng, J. Zhang, Urban traffic prediction from spatio-temporal data using deep meta learning, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, ACM, Anchorage AK USA, 2019, pp. 1720–1730, http://dx.doi.org/10.1145/3292500.3330884.

[68] Y. Qi, Q. Li, H. Karimian, D. Liu, A hybrid model for spatiotemporal forecasting of PM2.5 based on graph convolutional neural network and long short-term memory, Sci. Total Environ. 664 (2019) 1–10, http://dx.doi.org/10.1016/j.scitotenv.2019.01.333.

[69] S. Yang, W. Ma, X. Pi, S. Qian, A deep learning approach to real-time parking occupancy prediction in transportation networks incorporating multiple spatio-temporal data sources, Transp. Res. Part C Emerg. Technol. 107 (2019) 248–265, http://dx.doi.org/10.1016/j.trc.2019.08.010.

[70] L. Shi, Y. Zhang, J. Cheng, H. Lu, Two-stream adaptive graph convolutional networks for skeleton-based action recognition, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, in: CVPR, 2019, pp. 12026–12035, http://dx.doi.org/10.1109/CVPR.2019.01230.

[71] C. Zheng, X. Fan, C. Wang, J. Qi, GMAN: A graph multi-attention network for traffic prediction, in: Proceedings of the AAAI Conference on Artificial Intelligence, 34, (01) 2020, pp. 1234–1241, http://dx.doi.org/10.1609/aaai.v34i01.5477.

[72] C. Song, Y. Lin, S. Guo, H. Wan, Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, 34, (01) 2020, pp. 914–921, http://dx.doi.org/10.1609/aaai.v34i01.5438.

[73] L. Bai, L. Yao, C. Li, X. Wang, C. Wang, Adaptive graph convolutional recurrent network for traffic forecasting, in: H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, H. Lin (Eds.), in: Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 17804–17815, URL: https://proceedings.neurips.cc/paper/2020/hash/ce1aad92b939420fc17005e5461e6f48-Abstract.html.

[74] M. Li, Z. Zhu, Spatial-temporal fusion graph neural networks for traffic flow forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, 35, (5) 2021, pp. 4189–4196, http://dx.doi.org/10.1609/aaai.v35i5.16542.

[75] B. Yu, Y. Lee, K. Sohn, Forecasting road traffic speeds by considering area-wide spatio-temporal dependencies based on a graph convolutional neural network (GCN), Transp. Res. Part C Emerg. Technol. 114 (2020) 189–204, http://dx.doi.org/10.1016/j.trc.2020.02.013.

[76] B. Liu, E. Adeli, Z. Cao, K.-H. Lee, A. Shenoi, A. Gaidon, J.C. Niebles, Spatiotemporal relationship reasoning for pedestrian intent prediction, IEEE Robot. Autom. Lett. 5 (2) (2020) 3485–3492, http://dx.doi.org/10.1109/LRA.2020.2976305.

[77] Z. Cui, R. Ke, Z. Pu, X. Ma, Y. Wang, Learning traffic as a graph: A gated graph wavelet recurrent neural network for network-scale traffic prediction, Transp. Res. Part C Emerg. Technol. 115 (2020) 102620, http://dx.doi.org/10.1016/j.trc.2020.102620.

[78] R. Dai, S. Xu, Q. Gu, C. Ji, K. Liu, Hybrid spatio-temporal graph convolutional network: Improving traffic prediction with navigation data, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, ACM, Virtual Event CA USA, 2020, pp. 3074–3082, http://dx.doi.org/10.1145/3394486.3403358.

[79] H. Hong, Y. Lin, X. Yang, Z. Li, K. Fu, Z. Wang, X. Qie, J. Ye, HetETA: Heterogeneous information network embedding for estimating time of arrival, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, ACM, Virtual Event CA USA, 2020, pp. 2444–2454, http://dx.doi.org/10.1145/3394486.3403294.

[80] H.-W. Wang, Z.-R. Peng, D. Wang, Y. Meng, T. Wu, W. Sun, Q.-C. Lu, Evaluation and prediction of transportation resilience under extreme weather events: A diffusion graph convolutional approach, Transp. Res. Part C Emerg. Technol. 115 (2020) 102619, http://dx.doi.org/10.1016/j.trc.2020.102619.

[81] H. Shi, Q. Yao, Q. Guo, Y. Li, L. Zhang, J. Ye, Y. Li, Y. Liu, Predicting origin-destination flow via multi-perspective graph convolutional network, in: 2020 IEEE 36th International Conference on Data Engineering, ICDE, IEEE, Dallas, TX, USA, 2020, pp. 1818–1821, http://dx.doi.org/10.1109/ICDE48307.2020.00178.

[82] C. Yu, X. Ma, J. Ren, H. Zhao, S. Yi, Spatio-temporal graph transformer networks for pedestrian trajectory prediction, in: A. Vedaldi, H. Bischof, T. Brox, J.-M. Frahm (Eds.), in: Computer Vision – ECCV 2020, vol. 12357, Springer International Publishing, Cham, 2020, pp. 507–523, URL: https://link.springer.com/10.1007/978-3-030-58610-2_30.

[83] Z. Fang, Q. Long, G. Song, K. Xie, Spatial-temporal graph ODE networks for traffic flow forecasting, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, ACM, Virtual Event Singapore, 2021, pp. 364–373, http://dx.doi.org/10.1145/3447548.3467430.

[84] L. Yu, B. Du, X. Hu, L. Sun, L. Han, W. Lv, Deep spatio-temporal graph convolutional network for traffic accident prediction, Neurocomputing 423 (2021) 135–147, http://dx.doi.org/10.1016/j.neucom.2020.09.043.

[85] Y. Liu, J. Nie, X. Li, S.H. Ahmed, W.Y.B. Lim, C. Miao, Federated learning in the sky: Aerial-ground air quality sensing framework with UAV swarms, IEEE Internet Things J. 8 (12) (2021) 9827–9837, http://dx.doi.org/10.1109/JIOT.2020.3021006.

[86] X. Liu, M. Qin, Y. He, X. Mi, C. Yu, A new multi-data-driven spatiotemporal PM2.5 forecasting model based on an ensemble graph reinforcement learning convolutional network, Atmos. Pollut. Res. 12 (10) (2021) 101197, http://dx.doi.org/10.1016/j.apr.2021.101197.

[87] P. Zhang, J. Xu, Q. Wu, Y. Huang, X. Ben, Learning spatial-temporal representations over walking tracklet for long-term person Re-identification in the wild, IEEE Trans. Multimed. 23 (2021) 3562–3576, http://dx.doi.org/10.1109/TMM.2020.3028461.

[88] S. Eandi, A. Cini, S. Lukovic, C. Alippi, Spatio-temporal graph neural networks for aggregate load forecasting, in: 2022 International Joint Conference on Neural Networks, IJCNN, IEEE, Padua, Italy, 2022, pp. 1–8, http://dx.doi.org/10.1109/IJCNN55064.2022.9892780.

[89] S. Guo, Y. Lin, H. Wan, X. Li, G. Cong, Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting, IEEE Trans. Knowl. Data Eng. 34 (11) (2022) 5415–5428, http://dx.doi.org/10.1109/TKDE.2021.3056502.

[90] J. Zhu, X. Han, H. Deng, C. Tao, L. Zhao, P. Wang, T. Lin, H. Li, KST-GCN: A knowledge-driven spatial-temporal graph convolutional network for traffic forecasting, IEEE Trans. Intell. Transp. Syst. 23 (9) (2022) 15055–15065, http://dx.doi.org/10.1109/TITS.2021.3136287.

[91] Z. Shao, Z. Zhang, F. Wang, Y. Xu, Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, ACM, Washington DC USA, 2022, pp. 1567–1577, http://dx.doi.org/10.1145/3534678.3539396.

[92] X. Ta, Z. Liu, X. Hu, L. Yu, L. Sun, B. Du, Adaptive spatio-temporal graph neural network for traffic forecasting, Knowl.-Based Syst. 242 (2022) 108199, http://dx.doi.org/10.1016/j.knosys.2022.108199.

[93] Z. Pan, W. Zhang, Y. Liang, W. Zhang, Y. Yu, J. Zhang, Y. Zheng, Spatio-temporal meta learning for urban traffic prediction, IEEE Trans. Knowl. Data Eng. 34 (3) (2022) 1462–1476, http://dx.doi.org/10.1109/TKDE.2020.2995855.

[94] Z. Shao, Z. Zhang, W. Wei, F. Wang, Y. Xu, X. Cao, C.S. Jensen, Decoupled dynamic spatial-temporal graph neural network for traffic forecasting, Proc. VLDB Endow. 15 (11) (2022) 2733–2746, http://dx.doi.org/10.14778/3551793.3551827.

[95] L. Feng, C. Cheng, M. Zhao, H. Deng, Y. Zhang, EEG-based emotion recognition using spatial-temporal graph convolutional LSTM with attention mechanism, IEEE J. Biomed. Heal. Inform. 26 (11) (2022) 5406–5417, http://dx.doi.org/10.1109/JBHI.2022.3198688.

[96] J. Tang, J. Zeng, Spatiotemporal gated graph attention network for urban traffic flow prediction based on license plate recognition data, Comput. Aided Civ. Infrastruct. Eng. 37 (1) (2022) 3–23, http://dx.doi.org/10.1111/mice.12688.

[97] L. Wang, A. Adiga, J. Chen, A. Sadilek, S. Venkatramanan, M. Marathe, CausalGNN: Causal-based graph neural networks for spatio-temporal epidemic forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, 36, (11) 2022, pp. 12191–12199, http://dx.doi.org/10.1609/aaai.v36i11.21479.

[98] B. Yan, G. Wang, J. Yu, X. Jin, H. Zhang, Spatial-temporal Chebyshev graph neural network for traffic flow prediction in IoT-based ITS, IEEE Internet Things J. 9 (12) (2022) 9266–9279, http://dx.doi.org/10.1109/JIOT.2021.3105446.

[99] S.R. Vankdoth, M. Arock, Deep intelligent transportation system for travel time estimation on spatio-temporal data, Neural Comput. Appl. 35 (26) (2023) 19117–19129, http://dx.doi.org/10.1007/s00521-023-08726-3.

[100] C. Ding, S. Sun, J. Zhao, MST-GAT: A multimodal spatial–temporal graph attention network for time series anomaly detection, Inf. Fusion 89 (2023) 527–536, http://dx.doi.org/10.1016/j.inffus.2022.08.011.

[101] Z. Li, J. Yu, G. Zhang, L. Xu, Dynamic spatio-temporal graph network with adaptive propagation mechanism for multivariate time series forecasting, Expert Syst. Appl. 216 (2023) 119374, http://dx.doi.org/10.1016/j.eswa.2022.119374.

[102] R. Jiang, Z. Wang, J. Yong, P. Jeph, Q. Chen, Y. Kobayashi, X. Song, S. Fukushima, T. Suzumura, Spatio-temporal meta-graph learning for traffic forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, 37, (7) 2023, pp. 8078–8086, http://dx.doi.org/10.1609/aaai.v37i7.25976.

[103] C. Diao, D. Zhang, W. Liang, K.-C. Li, Y. Hong, J.-L. Gaudiot, A novel spatial-temporal multi-scale alignment graph neural network security model for vehicles prediction, IEEE Trans. Intell. Transp. Syst. 24 (1) (2023) 904–914, http://dx.doi.org/10.1109/TITS.2022.3140229.

[104] Y. Chen, K. Li, C.K. Yeo, K. Li, Traffic forecasting with graph spatial–temporal position recurrent network, Neural Netw. 162 (2023) 340–349, http://dx.doi.org/10.1016/j.neunet.2023.03.009.

[105] X. Huang, Y. Ye, X. Yang, L. Xiong, Multi-view dynamic graph convolution neural network for traffic flow prediction, Expert Syst. Appl. 222 (2023) 119779, http://dx.doi.org/10.1016/j.eswa.2023.119779.

[106] Z. Xie, G. Zheng, L. Miao, W. Huang, STGL-GCN: Spatial–temporal mixing of global and local self-attention graph convolutional networks for human action recognition, IEEE Access 11 (2023) 16526–16532, http://dx.doi.org/10.1109/ACCESS.2023.3246127.

[107] Y. Liu, T. Feng, S. Rasouli, M. Wong, ST-DAGCN: A spatiotemporal dual adaptive graph convolutional network model for traffic prediction, Neurocomputing 601 (2024) 128175, http://dx.doi.org/10.1016/j.neucom.2024.128175.

[108] P. Bikram, S. Das, A. Biswas, Dynamic attention aggregated missing spatial–temporal data imputation for traffic speed prediction, Neurocomputing 607 (2024) 128441, http://dx.doi.org/10.1016/j.neucom.2024.128441.

[109] W. Zhang, H. Wang, F. Zhang, Spatio-temporal Fourier enhanced heterogeneous graph learning for traffic forecasting, Expert Syst. Appl. 241 (2024) 122766, http://dx.doi.org/10.1016/j.eswa.2023.122766.

[110] Y. Chen, Y. Xie, X. Dang, B. Huang, C. Wu, D. Jiao, Spatiotemporal prediction of carbon emissions using a hybrid deep learning model considering temporal and spatial correlations, Environ. Model. Softw. 172 (2024) 105937, http://dx.doi.org/10.1016/j.envsoft.2023.105937.

[111] Y. Fang, Y. Qin, H. Luo, F. Zhao, K. Zheng, STWave+: A multi-scale efficient spectral graph attention network with long-term trends for disentangled traffic flow forecasting, IEEE Trans. Knowl. Data Eng. 36 (6) (2024) 2671–2685, http://dx.doi.org/10.1109/TKDE.2023.3324501.

[112] Y. Zhang, Y. Liang, J. Leng, Z. Wang, SCGTracker: Spatio-temporal correlation and graph neural networks for multiple object tracking, Pattern Recognit. 149 (2024) 110249, http://dx.doi.org/10.1016/j.patcog.2023.110249.

[113] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, H. Li, T-GCN: A temporal graph convolutional network for traffic prediction, IEEE Trans. Intell. Transp. Syst. 21 (9) (2020) 3848–3858, http://dx.doi.org/10.1109/TITS.2019.2935152.

[114] Y. Wang, C. Jing, S. Xu, T. Guo, Attention based spatiotemporal graph attention networks for traffic flow forecasting, Inform. Sci. 607 (2022) 869–883, http://dx.doi.org/10.1016/j.ins.2022.05.127.

[115] C. Wang, K. Zhang, H. Wang, B. Chen, Auto-STGCN: Autonomous spatial-temporal graph convolutional network search, ACM Trans. Knowl. Discov. Data 17 (5) (2023) 1–21, http://dx.doi.org/10.1145/3571285.

[116] Caltrans, Performance measurement system (PeMS), 2024, URL: https://pems.dot.ca.gov/.

[117] H.V. Jagadish, J. Gehrke, A. Labrinidis, Y. Papakonstantinou, J.M. Patel, R. Ramakrishnan, C. Shahabi, Big data and its technical challenges, Commun. ACM 57 (7) (2014) 86–94, http://dx.doi.org/10.1145/2611567.

[118] NYC TLC, TLC trip record data, 2024, URL: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[119] Citi Bike, NYC citi bike system data, 2024, URL: https://citibikenyc.com/system-data.

[120] National Renewable Energy Laboratory, Eastern wind integration dataset, 2012, URL: https://www.nrel.gov/grid/eastern-wind-data.html.

[121] Remote Sensing Systems, C. Mears, T. Lee, L. Ricciardulli, X. Wang, F. Wentz, RSS cross-calibrated multi-platform (CCMP) 6-hourly ocean vector wind analysis on 0.25 deg grid, version 3.0, 2022, http://dx.doi.org/10.56236/RSS-uv6h30.

[122] T. Oda, ODIAC Fossil Fuel CO2 Emissions Dataset, National Institute for Environmental Studies, 2015, http://dx.doi.org/10.17595/20170411.001.

[123] A. Shahroudy, J. Liu, T.-T. Ng, G. Wang, NTU RGB+D: A large scale dataset for 3D human activity analysis, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR, IEEE, Las Vegas, NV, USA, 2016, pp. 1010–1019, http://dx.doi.org/10.1109/CVPR.2016.115.

[124] J. Liu, A. Shahroudy, M. Perez, G. Wang, L.-Y. Duan, A.C. Kot, NTU RGB+D 120: A large-scale benchmark for 3D human activity understanding, IEEE Trans. Pattern Anal. Mach. Intell. 42 (10) (2020) 2684–2701, http://dx.doi.org/10.1109/TPAMI.2019.2916873.

[125] C. Ionescu, D. Papava, V. Olaru, C. Sminchisescu, Human3.6M: Large scale datasets and predictive methods for 3D human sensing in natural environments, IEEE Trans. Pattern Anal. Mach. Intell. 36 (7) (2014) 1325–1339, http://dx.doi.org/10.1109/TPAMI.2013.248.

[126] J. Carreira, A. Zisserman, Quo vadis, action recognition? A new model and the kinetics dataset, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR, 2017, pp. 6299–6308, URL: https://openaccess.thecvf.com/content_cvpr_2017/html/Carreira_Quo_Vadis_Action_CVPR_2017_paper.html.

[127] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev, M. Suleyman, A. Zisserman, The kinetics human action video dataset, 2017, http://dx.doi.org/10.48550/ARXIV.1705.06950.

[128] A. Rasouli, I. Kotseruba, J.K. Tsotsos, Are they going to cross? A benchmark dataset and baseline for pedestrian crosswalk behavior, in: 2017 IEEE International Conference on Computer Vision Workshops, ICCVW, IEEE, Venice, Italy, 2017, pp. 206–213, http://dx.doi.org/10.1109/ICCVW.2017.33.

[129] Stanford AI, Stanford-TRI intent prediction dataset, 2020, URL: https://stip.stanford.edu/dataset.html.

[130] Commission for Energy Regulation (CER), CER smart metering project - electricity customer behaviour trial, 2009–2010, 2012, URL: https://www.ucd.ie/issda/data/commissionforenergyregulationcer.

[131] National Renewable Energy Laboratory, Solar power data for integration studies, 2006, URL: https://www.nrel.gov/grid/solar-power-data.html.

[132] D.K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, R.P. Adams, Convolutional networks on graphs for learning molecular fingerprints, in: C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 28, Curran Associates, Inc., 2015, pp. 2224–2232, URL: https://papers.nips.cc/paper_files/paper/2015/hash/f9be311e65d81a9ad8150a60844bb94c-Abstract.html.

[133] J. Atwood, D. Towsley, Diffusion-convolutional neural networks, in: D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 29, Curran Associates, Inc., 2016, pp. 2001–2009, URL: https://papers.nips.cc/paper_files/paper/2016/hash/390e982518a50e280d8e2b535462ec1f-Abstract.html.

[134] F.R.K. Chung, Spectral Graph Theory, Regional conference series in mathematics, (92) Published for the Conference Board of the mathematical sciences by the American Mathematical Society, Providence, R.I, 1997.

[135] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, in: D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 29, Curran Associates, Inc., 2016, pp. 3844–3852, URL: https://papers.nips.cc/paper_files/paper/2016/hash/04df4d434d481c5bb723be1b6df1ee65-Abstract.html.

[136] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2016, http://dx.doi.org/10.48550/ARXIV.1609.02907.

[137] D.K. Hammond, P. Vandergheynst, R. Gribonval, Wavelets on graphs via spectral graph theory, Appl. Comput. Harmon. Anal. 30 (2) (2011) 129–150, http://dx.doi.org/10.1016/j.acha.2010.04.005.

[138] M. Balcilar, G. Renton, P. Heroux, B. Gauzere, S. Adam, P. Honeine, Bridging the gap between spectral and spatial domains in graph neural networks, 2020, http://dx.doi.org/10.48550/ARXIV.2003.11702.

[139] Z. Chen, F. Chen, L. Zhang, T. Ji, K. Fu, L. Zhao, F. Chen, L. Wu, C. Aggarwal, C.-T. Lu, Bridging the gap between spatial and spectral domains: A unified framework for graph neural networks, ACM Comput. Surv. 56 (5) (2024) 1–42, http://dx.doi.org/10.1145/3627816.

[140] Z. Chen, L. Zhang, L. Zhao, Unifying spectral and spatial graph neural networks, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, ACM, Boise ID USA, 2024, pp. 5511–5513, http://dx.doi.org/10.1145/3627673.3679088.

[141] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, 2014, http://dx.doi.org/10.48550/ARXIV.1409.0473, Publisher: arXiv Version Number: 7.

[142] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: International Conference on Learning Representations, 2018, URL: https://openreview.net/forum?id=rJXMpikCZ.

[143] M.-T. Luong, H. Pham, C.D. Manning, Effective approaches to attention-based neural machine translation, 2015, http://dx.doi.org/10.48550/ARXIV.1508.04025, Publisher: arXiv Version Number: 5.

[144] J. Zhang, X. Shi, J. Xie, H. Ma, I. King, D.-Y. Yeung, GaAN: Gated attention networks for learning on large and spatiotemporal graphs, 2018, http://dx.doi.org/10.48550/ARXIV.1803.07294.

[145] A. Cini, I. Marisca, F.M. Bianchi, C. Alippi, Scalable spatiotemporal graph neural networks, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, (6) 2023, pp. 7218–7226, http://dx.doi.org/10.1609/aaai.v37i6.25880.

[146] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, S. Sabato (Eds.), Proceedings of the 39th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, 162, PMLR, 2022, pp. 27268–27286, URL: https://proceedings.mlr.press/v162/zhou22g.html.

[147] Y. Zhang, J. Yan, Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: The Eleventh International Conference on Learning Representations, 2023, URL: https://openreview.net/forum?id=vSVLM2j9eie.

[148] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, Y. Sheikh, OpenPose: Realtime multi-person 2D pose estimation using part affinity fields, IEEE Trans. Pattern Anal. Mach. Intell. 43 (1) (2021) 172–186, http://dx.doi.org/10.1109/TPAMI.2019.2929257.

[149] D.J.C. MacKay, A practical Bayesian framework for backpropagation networks, Neural Comput. 4 (3) (1992) 448–472, http://dx.doi.org/10.1162/neco.1992.4.3.448.

[150] Y. Lipman, R.T.Q. Chen, H. Ben-Hamu, M. Nickel, M. Le, Flow matching for generative modeling, 2022, http://dx.doi.org/10.48550/ARXIV.2210.02747, Version Number: 2.

[151] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, in: H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, H. Lin (Eds.), in: Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 6840–6851, URL: https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html.

[152] J. Song, C. Meng, S. Ermon, Denoising diffusion implicit models, in: International Conference on Learning Representations, 2021, URL: https://openreview.net/forum?id=St1giarCHLP.

[153] Y. Yao, Y. Liu, X. Dai, S. Chen, Y. Lv, A graph-based scene encoder for vehicle trajectory prediction using the diffusion model, in: 2023 International Annual Conference on Complex Systems and Intelligent Science, CSIS-IAC, IEEE, Shenzhen, China, 2023, pp. 981–986, http://dx.doi.org/10.1109/CSIS-IAC60628.2023.10363970.

[154] H. Wen, Y. Lin, Y. Xia, H. Wan, Q. Wen, R. Zimmermann, Y. Liang, DiffSTG: Probabilistic spatio-temporal graph forecasting with denoising diffusion models, in: Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems, ACM, Hamburg Germany, 2023, pp. 1–12, http://dx.doi.org/10.1145/3589132.3625614.

[155] G. Liang, P. Tiwari, S.a. Nowaczyk, S. Byttner, F. Alonso-Fernandez, Dynamic causal explanation based diffusion-variational graph neural network for spatiotemporal forecasting, IEEE Trans. Neural Netw. Learn. Syst. (2024) 1–14, http://dx.doi.org/10.1109/TNNLS.2024.3415149.

[156] J. Chen, T. Ma, C. Xiao, FastGCN: Fast learning with graph convolutional networks via importance sampling, in: International Conference on Learning Representations, 2018, URL: https://openreview.net/forum?id=rytstxWAW.

[157] Y. Chen, T. Xu, D. Hakkani-Tur, D. Jin, Y. Yang, R. Zhu, Revisiting layer-wise sampling in fast training for graph convolutional networks, 2022, URL: https://openreview.net/forum?id=RRj7DcsPjT.

[158] Z. Shi, X. Liang, J. Wang, LMC: Fast training of GNNs via subgraph sampling with provable convergence, in: The Eleventh International Conference on Learning Representations, 2023, URL: https://openreview.net/forum?id=5VBBA91N6n.

[159] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: H. Wallach, H. Larochelle, A. Beygelzimer, F.d. Alché-Buc, E. Fox, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 32, Curran Associates, Inc., 2019, URL: https://proceedings.neurips.cc/paper_files/paper/2019/file/d80b7040b773199015de6d3b4293c8ff-Paper.pdf.

[160] J. Zhou, X. Qin, Y. Ding, H. Ma, Spatial–temporal dynamic graph differential equation network for traffic flow forecasting, Mathematics 11 (13) (2023) 2867, http://dx.doi.org/10.3390/math11132867.

Vincenzo Capone was born in Italy in 1997. He had a B.Sc. in Computer Science at University Parthenope of Naples in 2020 and a M.Sc. in Applied Computer Science (Machine Learning and Big Data) at University Parthenope of Naples in 2023. He currently is a second-year Ph.D. student in Artificial Intelligence at University Parthenope of Naples.

Angelo Casolaro was born in Italy in 1997. He had a B.Sc. in Computer Science at University Parthenope of Naples in 2020 and a M.Sc. in Applied Computer Science (Machine Learning and Big Data) at University Parthenope of Naples in 2022. He currently is a third-year Ph.D. student in Artificial Intelligence at University Parthenope of Naples.

Francesco Camastra was born in Italy in 1960. He had a M. Sc. in Physics at University of Milan in 1985 and a Ph.D. in Computer Science at University of Genoa in 2004. From February 2006 to December 28 2017 he was Assistant Professor in Computer Science at Parthenope University of Naples . Since December 29 2017, he has been Associate Professor of Computer Science at Parthenope University of Naples. He is the recipient of PR award 2008 from Pattern Recognition Journal and Eduardo R. Caianiello 2005 prize from Italian Neural Network Society. He is included by Plos Biology in the 100K top scientists. He has been Senior IEEE Member since December 27 2018.